

Dual Source Latent Demand Modeling for AI Search

Algorithms with Full Dry-Run Calculations

Gravton Labs — Parinaya Chaturvedi [parinayac@gravton.ai]

February 3, 2026

Abstract

This document provides two complete, self-contained estimation procedures: (i) Case 1 uses only an AI-search sensor (ASV) and performs Bayesian inference on the log-scale, then aggregates keyword-level demand to prompt-level demand via similarity-based mixture weights; (ii) Case 2 fuses ASV with classic search volume (SV) using an explicit coupling prior and closed-form Gaussian updates. Each case is written as a single section containing micro-steps, and each micro-step includes motivation, theory, parameter types (Observed/Calculated/Hyperparameters), explicit intermediate calculations, and a running numerical dry-run example.

Contents

1	Mathematical and Statistical Prerequisites	4
1.1	What problem are we solving (in plain math words)?	4
1.2	Notation and types of quantities	4
1.3	Log transform and why it is used	5
1.4	Random variables and probability distributions used	5
1.4.1	Gaussian / Normal distribution	5
1.4.2	Lognormal distribution	5
1.5	Core Bayesian inference concepts	6
1.5.1	Bayes' Rule	6
1.5.2	Conjugacy in the Gaussian model	6
1.6	Likelihood models for sensors	6
1.6.1	Single-sensor Gaussian observation (Case 1)	6
1.6.2	Two-sensor model (Case 2) and coupling	6
1.7	Weighted aggregation and similarity	7
1.8	Estimation methods referenced by the algorithm	7
1.8.1	MAP vs posterior mean	7
1.9	Error propagation and uncertainty reporting	7
1.10	Checklist: after reading this section, you should be able to	8
2	Mathematical and Statistical Derivations	8
2.1	Derivation A: Case 1 posterior for one keyword with ASV only	8
2.1.1	Model definition (single keyword k)	8
2.1.2	Posterior derivation using “complete the square”	8
2.2	Derivation B: Converting log-posterior to demand on original scale	9
2.3	Derivation C: Similarity weights and why softmax normalization appears	9
2.4	Derivation D: Aggregation on log-scale vs original scale	10
2.4.1	Option 1: Aggregate in original space	10
2.4.2	Option 2: Aggregate in log-space	10

2.5	Derivation A-D (a) : Empirical Estimation of Class- c Hyperparameters for Case 1 (ASV-only) [using above derivations]	10
2.5.1	D.1 Data required (observables)	11
2.5.2	D.2 Estimating the ASV log-bias b_A (or $b_{A,c}$)	11
2.5.3	D.3 Estimating sensor measurement noise $\sigma_{A,c}^2$ from the ASV time series	11
2.5.4	D.4 Estimating the class prior (μ_c, τ_c^2) (Empirical Bayes, explicit)	12
2.5.5	D.5 Plug-in to Case 1 posterior (no algorithm change)	13
2.6	Derivation E: Case 2 Fusion of ASV and SV as Two Gaussian Measurements	13
2.6.1	E.1 Motivation and modeling assumptions	13
2.6.2	E.2 Model specification (prior + likelihoods)	13
2.6.3	E.3 Posterior derivation from first principles (no “magic steps”)	14
2.6.4	E.4 Estimating the offset δ (when not fixed)	16
2.7	Derivation F: Coupling-Prior Variant (Two Latents with a Soft Constraint)	16
2.7.1	F.1 Model and meaning of the coupling prior	16
2.7.2	F.2 Deriving the MAP / posterior mean via quadratic optimization	17
2.7.3	F.2.0 Where does the negative log-posterior come from? What does “up to constants” mean?	18
2.7.4	F.3 Relationship between Derivation E and Derivation F	20
2.7.5	F.4 Practical guidance: how to choose / estimate σ_C^2 and δ	20
2.8	Derivation G: From keyword-level posteriors to prompt-level posterior (uncertainty)	21
2.9	Derivation H: Why “observed vs calculated vs hyperparameters” matters mathematically	21
2.10	End goal: after reading derivations, you should be able to re-derive every formula in Case 1 and Case 2	21
3	Case 1 (Only ASV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)	22
4	Case 2 (ASV + SV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)	29
5	FAQs	43
5.1	1) “Are we not using hierarchical shrinkage anywhere?”	43
5.1.1	Yes, we are — it is exactly what happens in the conjugate Bayesian update (Steps 9–11).	43
5.2	2) “Are we not estimating variances using time series of observed sensors?”	44
5.2.1	Yes — that is exactly what $\sigma_{A,c}^2$ (and $\sigma_{S,c}^2$ in Case 2) and often σ_{drift}^2 are for.	44
5.2.2	(A) Measurement noise variance (sensor noise): $\sigma_{A,c}^2, \sigma_{S,c}^2$	44
5.2.3	(B) Drift / process variance: σ_{drift}^2	45
5.3	3) “How exactly are we using keyword-intent c ? Where does it fit in both cases?”	45
5.3.1	c is not cosmetic — it is structurally required for pooling and class-conditional calibration.	45
5.3.2	(i) Class-conditioned priors (hierarchical pooling)	45
5.3.3	(ii) Class-conditioned sensor noise	45
5.3.4	(iii) Case 2: coupling “classic demand $\rightarrow$ AI demand” is usually class-dependent	46
6	Future Scope: Gaps, Risks, and Concrete Fixes (Production-Readiness Roadmap)	46
6.1	1) Hierarchical Shrinkage is Present but Under-specified	46
6.2	2) Temporal Dynamics and Non-Stationarity are Missing	47
6.3	3) Sensor Calibration and Cross-Validation Framework is Missing	48
6.4	4) Sparsity, Missingness, and Cold-Start are Not Addressed	48

6.5	5) Correlation Structure Between Keywords is Ignored	49
6.6	6) Measurement Error Model is Oversimplified (Heteroskedasticity, Bias, Latency)	49
6.7	7) The ρ (AI-share) Model Needs Stronger Justification and Estimation	50
6.8	8) Aggregation Method $\hat{Y}(p) = \sum_k w_{p,k} \hat{A}(k)$ Needs Justification	50
6.9	9) Uncertainty Propagation is Incomplete	51
6.10	10) Case 1 vs. Case 2 Separation Could Be Unified	51
6.11	11) “Coupling Prior” in Case 2 is Essentially a Regression and May Need Flexibility	51
6.12	12) Operational, Monitoring, and Governance Gaps	52

1. Mathematical and Statistical Prerequisites

1.1 What problem are we solving (in plain math words)?

We observe *noisy proxy sensors* that correlate with an unobserved quantity we truly care about:

latent monthly demand θ for a prompt/topic/intent.

Instead of observing θ directly, we observe:

- **ASV** (AI-search sensor value) for keywords, denoted $ASV(k)$.
- (Case 2 only) **SV** (classic search volume) for keywords, denoted $SV(k)$.

The core modeling idea is:

$$\underbrace{\log(\text{sensor})}_{\text{observed}} \approx \underbrace{\log(\text{latent demand})}_{\text{unknown}} + \underbrace{\text{noise}}_{\text{random}}.$$

Working on the log-scale turns multiplicative noise into additive noise, and makes positive quantities easier to model.

1.2 Notation and types of quantities

We use three **types** of quantities throughout Case 1 and Case 2.

Observed (O)

These are measured or externally provided:

- Keywords $k \in \mathcal{K}$.
- Observed AI-search sensor values $ASV(k)$.
- (Case 2) Observed search volumes $SV(k)$.
- Text embeddings (or similarity scores) used to relate keyword k to prompt/topic p .

Calculated (C)

These are computed inside the algorithm:

- Log-sensor values: $y_A(k) = \log ASV(k)$, $y_S(k) = \log SV(k)$.
- Keyword-level posterior parameters: means and variances for latent log-demand.
- Similarity weights and normalized mixture weights.
- Prompt-level aggregated latent demand.

Hyperparameters (H)

These are modeling choices you set:

- Prior mean and variance for latent log-demand.
- Noise variances of sensors on the log-scale (ASV noise, SV noise).
- Coupling strength between ASV and SV latent signals (Case 2).
- Similarity temperature or scaling, smoothing constants, and any regularization.

1.3 Log transform and why it is used

If a quantity $X > 0$ varies multiplicatively (common in volumes),

$$X = \theta \cdot \varepsilon,$$

where $\varepsilon > 0$ is multiplicative noise, then taking logs yields

$$\log X = \log \theta + \log \varepsilon.$$

If $\log \varepsilon$ is approximately normal, we get a linear Gaussian model, which leads to:

- closed-form Bayesian updates (no sampling required),
- numerically stable computations,
- interpretability: additive decompositions on the log-scale correspond to multiplicative decompositions on the original scale.

1.4 Random variables and probability distributions used

1.4.1 Gaussian / Normal distribution

A random variable Z is normal:

$$Z \sim \mathcal{N}(\mu, \sigma^2)$$

with density

$$p(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z - \mu)^2}{2\sigma^2}\right).$$

Key facts:

- Sum of independent normals is normal.
- Posterior in a linear-Gaussian model remains Gaussian (conjugacy).

1.4.2 Lognormal distribution

If

$$Z \sim \mathcal{N}(\mu, \sigma^2),$$

then

$$X = \exp(Z) \sim \text{LogNormal}(\mu, \sigma^2).$$

Useful identities:

$$\mathbb{E}[X] = \exp\left(\mu + \frac{\sigma^2}{2}\right), \quad \text{Median}(X) = \exp(\mu), \quad \text{Mode}(X) = \exp(\mu - \sigma^2).$$

This matters because latent demand is positive. If we infer $Z = \log \theta$ as Gaussian, then $\theta = \exp(Z)$ is lognormal.

1.5 Core Bayesian inference concepts

1.5.1 Bayes' Rule

For parameter θ and observed data D :

$$p(\theta | D) = \frac{p(D | \theta) p(\theta)}{p(D)}.$$

Interpretation:

- $p(\theta)$: **prior** belief before seeing data.
- $p(D | \theta)$: **likelihood** how data is generated from θ .
- $p(\theta | D)$: **posterior** belief after seeing data.

1.5.2 Conjugacy in the Gaussian model

If the prior is Gaussian and the likelihood is Gaussian with linear relationship, the posterior is Gaussian with *closed-form* formulas. This is why Case 1 and Case 2 are designed to be linear-Gaussian on the log-scale.

Note: A Linear Gaussian model (or system) defines a relationship where variables follow a Gaussian (normal) distribution, and the mean of the dependent variable is a linear function of its parent variables, plus Gaussian noise. Commonly used in regression and Kalman filters, this structure allows for efficient, exact inference because linear transformations of Gaussian variables remain Gaussian. Structure: The model is generally expressed as $x = Az + \epsilon$, where x is the observed data, z is the latent/parent variable, A is a transformation matrix, and $\epsilon \sim N(0, \Sigma)$ is Gaussian noise.

1.6 Likelihood models for sensors

1.6.1 Single-sensor Gaussian observation (Case 1)

Let $z_k = \log \theta_k$ be latent log-demand for keyword k . We observe the log ASV sensor:

$$y_A(k) = \log ASV(k).$$

Model:

$$y_A(k) | z_k \sim \mathcal{N}(z_k, \sigma_A^2).$$

Here σ_A^2 is a hyperparameter: sensor noise on log-scale.

1.6.2 Two-sensor model (Case 2) and coupling

In Case 2 we have

$$y_A(k) = \log ASV(k), \quad y_S(k) = \log SV(k).$$

A common structure is:

$$y_A(k) | z_k \sim \mathcal{N}(z_k, \sigma_A^2), \quad y_S(k) | z_k \sim \mathcal{N}(z_k + \delta, \sigma_S^2),$$

where δ is a calibration offset capturing systematic differences between SV and ASV scales. Alternatively, we can use a **coupled latent** formulation with a coupling prior; the derivations for both appear in the derivations section.

1.7 Weighted aggregation and similarity

We often need to map keyword-level estimates to prompt/topic-level estimates. We use similarity-based weights:

$$s_{p,k} \in [0, 1] \quad (\text{similarity between prompt } p \text{ and keyword } k).$$

A common conversion to weights is a softmax / normalized exponential:

$$w_{p,k} = \frac{\exp(\tau s_{p,k})}{\sum_{j \in \mathcal{K}} \exp(\tau s_{p,j})},$$

where $\tau > 0$ is a hyperparameter (**temperature/scale**) controlling sharpness.

Aggregation examples:

- On **log-scale** (mixture of log-demands):

$$\hat{z}_p = \sum_k w_{p,k} \hat{z}_k.$$

- On **original scale** (mixture of demands):

$$\hat{\theta}_p = \sum_k w_{p,k} \hat{\theta}_k.$$

Important: averaging in log-space is not the same as averaging in original space. We will derive and interpret both choices in the derivations section.

1.8 Estimation methods referenced by the algorithm

1.8.1 MAP vs posterior mean

For Gaussian posteriors:

$$z_k \mid D \sim \mathcal{N}(\mu_k, v_k).$$

Then:

$$\text{posterior mean} = \mu_k, \quad \text{MAP} = \mu_k,$$

so they coincide. On original scale ($\theta_k = \exp(z_k)$), mean and median differ:

$$\text{Median}(\theta_k) = \exp(\mu_k), \quad \mathbb{E}[\theta_k] = \exp\left(\mu_k + \frac{v_k}{2}\right).$$

This is crucial when you convert back from log-space.

1.9 Error propagation and uncertainty reporting

If $z \sim \mathcal{N}(\mu, v)$ and $\theta = \exp(z)$, then θ is lognormal. A $(1 - \alpha)$ credible interval for z :

$$[\mu - z_{\alpha/2}\sqrt{v}, \mu + z_{\alpha/2}\sqrt{v}],$$

maps to a multiplicative interval for θ :

$$[\exp(\mu - z_{\alpha/2}\sqrt{v}), \exp(\mu + z_{\alpha/2}\sqrt{v})].$$

This is why uncertainty on log-scale translates into multiplicative uncertainty on demand.

1.10 Checklist: after reading this section, you should be able to

1. Explain why we use log-transform and Gaussian noise.
2. Write Bayes' rule and interpret prior/likelihood/posterior.
3. Derive (or follow) Gaussian conjugate updates.
4. Convert from Gaussian on log-scale to lognormal on original scale.
5. Understand similarity-based weighting and its hyperparameters.
6. Understand how uncertainty propagates through exponentiation.

2. Mathematical and Statistical Derivations

2.1 Derivation A: Case 1 posterior for one keyword with ASV only

2.1.1 Model definition (single keyword k)

Latent variable (unknown).

$$z_k \equiv \log \theta_k.$$

Prior (hyperparameters).

$$z_k \sim \mathcal{N}(\mu_0, v_0).$$

Observation (observed).

$$y_A(k) \equiv \log ASV(k).$$

Likelihood (hyperparameter σ_A^2).

$$y_A(k) \mid z_k \sim \mathcal{N}(z_k, \sigma_A^2).$$

2.1.2 Posterior derivation using “complete the square”

Bayes:

$$p(z_k \mid y_A) \propto p(y_A \mid z_k) p(z_k).$$

Write both densities up to proportionality:

$$p(y_A \mid z_k) \propto \exp\left(-\frac{(y_A - z_k)^2}{2\sigma_A^2}\right), \quad p(z_k) \propto \exp\left(-\frac{(z_k - \mu_0)^2}{2v_0}\right).$$

Multiply:

$$p(z_k \mid y_A) \propto \exp\left(-\frac{(y_A - z_k)^2}{2\sigma_A^2} - \frac{(z_k - \mu_0)^2}{2v_0}\right).$$

Expand the squares (showing all intermediate algebra):

$$(y_A - z_k)^2 = z_k^2 - 2y_A z_k + y_A^2, \quad (z_k - \mu_0)^2 = z_k^2 - 2\mu_0 z_k + \mu_0^2.$$

Substitute:

$$\begin{aligned} & -\frac{1}{2\sigma_A^2}(z_k^2 - 2y_A z_k + y_A^2) - \frac{1}{2v_0}(z_k^2 - 2\mu_0 z_k + \mu_0^2) \\ &= -\frac{1}{2}\left(\frac{1}{\sigma_A^2} + \frac{1}{v_0}\right)z_k^2 + \left(\frac{y_A}{\sigma_A^2} + \frac{\mu_0}{v_0}\right)z_k - \frac{1}{2}\left(\frac{y_A^2}{\sigma_A^2} + \frac{\mu_0^2}{v_0}\right). \end{aligned}$$

The last term does not depend on z_k and can be absorbed into the normalizing constant. So the posterior is proportional to:

$$\exp\left(-\frac{1}{2}az_k^2 + bz_k\right), \quad \text{where } a = \left(\frac{1}{\sigma_A^2} + \frac{1}{v_0}\right), \quad b = \left(\frac{y_A}{\sigma_A^2} + \frac{\mu_0}{v_0}\right).$$

Complete the square:

$$-\frac{1}{2}az_k^2 + bz_k = -\frac{1}{2}a\left(z_k^2 - 2\frac{b}{a}z_k\right) = -\frac{1}{2}a\left[\left(z_k - \frac{b}{a}\right)^2 - \left(\frac{b}{a}\right)^2\right].$$

Thus:

$$p(z_k | y_A) \propto \exp\left(-\frac{1}{2}a\left(z_k - \frac{b}{a}\right)^2\right),$$

which is a Gaussian with:

$$v_k = \frac{1}{a} = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2}\right)^{-1}, \quad \mu_k = \frac{b}{a} = v_k \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2}\right).$$

Interpretation (precision-weighted average). Define **precision** as $\pi = \frac{1}{\text{variance}}$. Then

$$\mu_k = \frac{\pi_0\mu_0 + \pi_A y_A}{\pi_0 + \pi_A}, \quad v_k = \frac{1}{\pi_0 + \pi_A},$$

where $\pi_0 = 1/v_0$ and $\pi_A = 1/\sigma_A^2$.

2.2 Derivation B: Converting log-posterior to demand on original scale

Given

$$z_k | D \sim \mathcal{N}(\mu_k, v_k), \quad \theta_k = \exp(z_k),$$

then θ_k is lognormal. Key outputs:

$$\text{Median}(\theta_k) = \exp(\mu_k), \quad \mathbb{E}[\theta_k] = \exp\left(\mu_k + \frac{v_k}{2}\right).$$

Which should the algorithm use?

- If you want a **typical** value robust to skew: use $\exp(\mu_k)$ (median).
- If you want the **expected** volume (mean): use $\exp(\mu_k + v_k/2)$.

Both are mathematically correct; they answer different questions. Your Case 1/Case 2 algorithms should state explicitly which one they output.

2.3 Derivation C: Similarity weights and why softmax normalization appears

Let $s_{p,k}$ be similarity between prompt/topic p and keyword k . We want nonnegative weights that sum to 1:

$$w_{p,k} \geq 0, \quad \sum_k w_{p,k} = 1.$$

A standard construction is:

$$w_{p,k} = \frac{\exp(\tau s_{p,k})}{\sum_j \exp(\tau s_{p,j})}.$$

Why exponentials? Exponentials convert additive differences in similarity into multiplicative ratios:

$$\frac{w_{p,k_1}}{w_{p,k_2}} = \exp(\tau(s_{p,k_1} - s_{p,k_2})).$$

So if s_{p,k_1} is bigger than s_{p,k_2} by Δ , then k_1 is weighted $\exp(\tau\Delta)$ times more than k_2 .

Role of τ (hyperparameter).

- Large $\tau \Rightarrow$ weights become *peaky* (nearly one-hot).
- Small $\tau \Rightarrow$ weights become *uniform*.

2.4 Derivation D: Aggregation on log-scale vs original scale

Suppose keyword-level latent demands are random variables $\theta_k = \exp(z_k)$. A prompt-level demand is often approximated by a weighted combination.

2.4.1 Option 1: Aggregate in original space

$$\theta_p \approx \sum_k w_{p,k} \theta_k.$$

If we use posterior mean estimates:

$$\hat{\theta}_p = \sum_k w_{p,k} \mathbb{E}[\theta_k | D] = \sum_k w_{p,k} \exp\left(\mu_k + \frac{v_k}{2}\right).$$

This has a direct “expected volume” interpretation.

2.4.2 Option 2: Aggregate in log-space

$$z_p \approx \sum_k w_{p,k} z_k, \quad \Rightarrow \quad \hat{z}_p = \sum_k w_{p,k} \mu_k.$$

If you then output a typical demand:

$$\hat{\theta}_p = \exp(\hat{z}_p),$$

this corresponds to a kind of *geometric averaging*.

Important warning. In general,

$$\exp\left(\sum_k w_k \mu_k\right) \neq \sum_k w_k \exp(\mu_k).$$

So you must choose the aggregation domain intentionally and keep it consistent with what you claim to estimate.

For our Case 1 and 2, we will be using Option 1, but this is subject to experiments.

2.5 Derivation A-D (a) : Empirical Estimation of Class- c Hyperparameters for Case 1 (ASV-only) [using above derivations]

Goal. Case 1 uses class-conditioned hyperparameters

$$(\mu_c, \tau_c^2, b_A, \sigma_{A,c}^2)$$

in the Bayesian model

$$a_k \sim \mathcal{N}(\mu_c, \tau_c^2), \quad x_{k,t} | a_{k,t} \sim \mathcal{N}(a_{k,t} + b_A, \sigma_{A,c}^2), \quad x_{k,t} \triangleq \log(\text{ASV}_{k,t}).$$

This derivation shows how to compute these quantities explicitly from historical sensor time series, without assuming values.

Why classes exist (hierarchical pooling). The class index $c = c(k)$ defines pooling groups

$$K_c \triangleq \{k : c(k) = c\}, \quad |K_c| = n_c,$$

so that keywords in the same intent/semantic family share statistical properties:

- prior center and dispersion (μ_c, τ_c^2) (between-keyword variability in true demand),
- sensor measurement noise $\sigma_{A,c}^2$ (how noisy ASV is for this intent class),
- and optionally sensor bias b_A (systematic log-offset).

This is the mechanism that induces hierarchical shrinkage in the posterior.

2.5.1 D.1 Data required (observables)

Let $t = 1, \dots, T$ index months (or other fixed intervals). For each keyword $k \in K_c$, observe the ASV sensor:

$$\text{ASV}_{k,t} \quad (\text{Observed}), \quad x_{k,t} \triangleq \log(\text{ASV}_{k,t}) \quad (\text{Calculated}).$$

2.5.2 D.2 Estimating the ASV log-bias b_A (or $b_{A,c}$)

Model identity. From

$$x_{k,t} \approx a_{k,t} + b_A + \varepsilon_{k,t}, \quad \varepsilon_{k,t} \sim \mathcal{N}(0, \sigma_{A,c}^2),$$

bias is the systematic difference between the sensor and a truth-aligned quantity. Therefore, to estimate b_A we need a calibration proxy $g_{k,t}$ such that

$$g_{k,t} \approx a_{k,t} = \log(A_{k,t}^*) \quad \text{for a calibration subset of keyword-time pairs.}$$

Calibration set. Let $\mathcal{C} \subseteq \{(k, t)\}$ denote the set of calibration pairs with such a proxy, and let $|\mathcal{C}| = N_{\mathcal{C}}$.

Bias estimator (explicit). Using $x_{k,t} - g_{k,t} \approx b_A$, the unbiased estimator is

$$\hat{b}_A = \frac{1}{N_{\mathcal{C}}} \sum_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

Robust alternative:

$$\hat{b}_A^{\text{robust}} = \text{median}_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

If bias differs by class, estimate within class:

$$\hat{b}_{A,c} = \frac{1}{N_{\mathcal{C},c}} \sum_{(k,t) \in \mathcal{C} \cap (K_c \times \{1, \dots, T\})} (x_{k,t} - g_{k,t}).$$

2.5.3 D.3 Estimating sensor measurement noise $\sigma_{A,c}^2$ from the ASV time series

Baseline + residual principle. To estimate measurement noise, compute deviations around an estimate of the keyword's underlying level.

Per-keyword baseline (explicit). The simplest baseline is the time-mean:

$$\bar{x}_k \triangleq \frac{1}{T} \sum_{t=1}^T x_{k,t}.$$

Then compute the per-keyword residual variance:

$$\hat{\sigma}_A^2(k) \triangleq \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2.$$

Pooling to the class level (explicit). Mean pooling:

$$\hat{\sigma}_{A,c}^2 \triangleq \frac{1}{n_c} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

Robust pooling:

$$\hat{\sigma}_{A,c,\text{robust}}^2 \triangleq \text{median}_{k \in K_c} \hat{\sigma}_A^2(k).$$

If drift/seasonality exists (still explicit). If the latent $a_{k,t}$ changes materially over the window, residuals around $\bar{x}_k$ include drift. A principled fix is detrending via a smoothing operator:

$$\hat{m}_{k,t} = \text{Smooth}(x_{k,1:T})_t, \quad r_{k,t} = x_{k,t} - \hat{m}_{k,t},$$

then

$$\hat{\sigma}_A^2(k) = \frac{1}{T-1} \sum_{t=1}^T r_{k,t}^2, \quad \hat{\sigma}_{A,c}^2 = \frac{1}{n_c} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

2.5.4 D.4 Estimating the class prior (μ_c, τ_c^2) (Empirical Bayes, explicit)

Keyword-level level estimate (explicit). First de-bias and average to estimate each keyword's latent log-level:

$$\hat{a}_k \triangleq \frac{1}{T} \sum_{t=1}^T (x_{k,t} - \hat{b}_A) = \bar{x}_k - \hat{b}_A.$$

This is a noisy estimate of the keyword's latent mean level.

Class prior mean (explicit).

$$\hat{\mu}_c \triangleq \frac{1}{n_c} \sum_{k \in K_c} \hat{a}_k.$$

Robust alternative:

$$\hat{\mu}_c^{\text{robust}} = \text{median}_{k \in K_c} \hat{a}_k.$$

Class between-keyword variance (explicit with noise removal). Across keywords, the sample variance of $\hat{a}_k$ mixes true between-keyword variance τ_c^2 and estimation noise from averaging T measurements. Since

$$\hat{a}_k = \bar{x}_k - \hat{b}_A, \quad \text{Var}(\bar{x}_k) \approx \frac{\sigma_{A,c}^2}{T},$$

we have approximately

$$\text{Var}(\hat{a}_k) \approx \tau_c^2 + \frac{\sigma_{A,c}^2}{T}.$$

Hence an explicit Empirical-Bayes estimator is

$$\hat{\tau}_c^2 \triangleq \max \left\{ \underbrace{\frac{1}{n_c - 1} \sum_{k \in K_c} (\hat{a}_k - \hat{\mu}_c)^2}_{\text{sample variance across keywords}} - \underbrace{\frac{\hat{\sigma}_{A,c}^2}{T}}_{\text{remove averaging noise}}, \epsilon \right\},$$

where $\epsilon > 0$ is a small variance floor to ensure non-negativity.

2.5.5 D.5 Plug-in to Case 1 posterior (no algorithm change)

Once

$$(\hat{\mu}_c, \hat{\tau}_c^2, \hat{b}_A, \hat{\sigma}_{A,c}^2)$$

are computed, Case 1 uses exactly the same closed-form Normal–Normal posterior:

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\hat{\tau}_c^2} + \frac{1}{\hat{\sigma}_{A,c}^2} \right)^{-1}, \quad \mu_{k,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\hat{\mu}_c}{\hat{\tau}_c^2} + \frac{x_k - \hat{b}_A}{\hat{\sigma}_{A,c}^2} \right),$$

followed by lognormal back-transform and prompt-level mixing as already specified in Case 1.

2.6 Derivation E: Case 2 Fusion of ASV and SV as Two Gaussian Measurements

This derivation is the canonical *conjugate Gaussian update* when we have *two conditionally independent noisy measurements* of the same latent quantity. We will re-derive every step (including why “precisions add”) and also interpret the result.

2.6.1 E.1 Motivation and modeling assumptions

Goal. For a keyword k , we want to estimate the latent *true* log-demand

$$z_k \triangleq \log \theta_k,$$

using two observed sensors:

$$y_A \triangleq \log(\text{ASV}(k)), \quad y_S \triangleq \log(\text{SV}(k)).$$

Key assumptions.

1. **Latent truth exists.** There is a single underlying true log-demand z_k .
2. **Gaussian sensor noise in log-space.** Each sensor equals truth plus noise (in log scale).
3. **Conditional independence given truth.** Once z_k is fixed, the two sensors’ noises are independent:

$$p(y_A, y_S \mid z_k) = p(y_A \mid z_k) p(y_S \mid z_k).$$

This is the only assumption needed for the “precision additivity” result.

2.6.2 E.2 Model specification (prior + likelihoods)

Prior (before seeing sensors). We assume a Gaussian prior for the latent log-demand:

$$z_k \sim \mathcal{N}(\mu_0, v_0),$$

where:

- μ_0 is the prior mean (could be global or class-conditioned; here we write a single prior for clarity).
- v_0 is the prior variance (uncertainty before observing sensors).

ASV likelihood. We model the ASV log-sensor as:

$$y_A \mid z_k \sim \mathcal{N}(z_k, \sigma_A^2),$$

meaning:

$$y_A = z_k + \varepsilon_A, \quad \varepsilon_A \sim \mathcal{N}(0, \sigma_A^2).$$

SV likelihood with calibration offset. In practice SV often differs from ASV by a systematic log-scale offset δ (a multiplicative factor in linear scale). We model:

$$y_S \mid z_k \sim \mathcal{N}(z_k + \delta, \sigma_S^2),$$

meaning:

$$y_S = z_k + \delta + \varepsilon_S, \quad \varepsilon_S \sim \mathcal{N}(0, \sigma_S^2).$$

Aligned SV observation (removing bias). Define the bias-corrected SV log-observation:

$$\tilde{y}_S \triangleq y_S - \delta.$$

Then:

$$\tilde{y}_S \mid z_k \sim \mathcal{N}(z_k, \sigma_S^2),$$

so now both sensors are centered at the same latent z_k .

2.6.3 E.3 Posterior derivation from first principles (no “magic steps”)

Posterior definition. By Bayes’ rule:

$$p(z_k \mid y_A, \tilde{y}_S) \propto p(z_k) p(y_A \mid z_k) p(\tilde{y}_S \mid z_k).$$

Each term is Gaussian in z_k , so the product is proportional to a Gaussian in z_k . We will show this by expanding the *log posterior* and completing the square.

Write each density up to proportionality in z_k . Ignoring constants that do not depend on z_k :

$$\begin{aligned} p(z_k) &\propto \exp\left(-\frac{1}{2v_0}(z_k - \mu_0)^2\right), \\ p(y_A \mid z_k) &\propto \exp\left(-\frac{1}{2\sigma_A^2}(y_A - z_k)^2\right), \\ p(\tilde{y}_S \mid z_k) &\propto \exp\left(-\frac{1}{2\sigma_S^2}(\tilde{y}_S - z_k)^2\right). \end{aligned}$$

Multiply them (add exponents). Thus:

$$p(z_k \mid y_A, \tilde{y}_S) \propto \exp\left(-\frac{1}{2}\left[\frac{(z_k - \mu_0)^2}{v_0} + \frac{(y_A - z_k)^2}{\sigma_A^2} + \frac{(\tilde{y}_S - z_k)^2}{\sigma_S^2}\right]\right).$$

Define the negative log-posterior (up to constants):

$$\mathcal{L}(z_k) \triangleq \frac{(z_k - \mu_0)^2}{v_0} + \frac{(y_A - z_k)^2}{\sigma_A^2} + \frac{(\tilde{y}_S - z_k)^2}{\sigma_S^2}.$$

Expand $\mathcal{L}(z_k)$ into a quadratic in z_k . Expand each square:

$$(z_k - \mu_0)^2 = z_k^2 - 2\mu_0 z_k + \mu_0^2,$$

$$(y_A - z_k)^2 = z_k^2 - 2y_A z_k + y_A^2,$$

$$(\tilde{y}_S - z_k)^2 = z_k^2 - 2\tilde{y}_S z_k + \tilde{y}_S^2.$$

Substitute:

$$\mathcal{L}(z_k) = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \right) z_k^2 - 2 \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \right) z_k + (\text{constants}).$$

Let

$$\pi_k \triangleq \frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \quad (\text{posterior precision}),$$

$$b_k \triangleq \frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \quad (\text{precision-weighted sum}).$$

Then:

$$\mathcal{L}(z_k) = \pi_k z_k^2 - 2b_k z_k + (\text{constants}).$$

Complete the square to identify the Gaussian posterior. We rewrite:

$$\pi_k z_k^2 - 2b_k z_k = \pi_k \left[z_k^2 - 2 \left(\frac{b_k}{\pi_k} \right) z_k \right] = \pi_k \left[\left(z_k - \frac{b_k}{\pi_k} \right)^2 - \left(\frac{b_k}{\pi_k} \right)^2 \right].$$

Thus (up to constants):

$$p(z_k \mid y_A, \tilde{y}_S) \propto \exp \left(-\frac{\pi_k}{2} \left(z_k - \frac{b_k}{\pi_k} \right)^2 \right),$$

which is exactly a Gaussian with:

$$v_k = \frac{1}{\pi_k}, \quad \mu_k = \frac{b_k}{\pi_k}.$$

Substituting b_k back:

$$v_k = \left(\frac{1}{v_0} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_S^2} \right)^{-1}$$

$$\mu_k = v_k \left(\frac{\mu_0}{v_0} + \frac{y_A}{\sigma_A^2} + \frac{\tilde{y}_S}{\sigma_S^2} \right).$$

Why “precisions add”? (Statistical intuition) Each Gaussian term contributes a quadratic penalty in z_k :

$$\frac{(z_k - \text{center})^2}{\text{variance}}.$$

Quadratic penalties add; therefore the coefficients of z_k^2 add. Those coefficients are precisely the *precisions* (1/variance). Hence precision additivity is just “adding curvature” of independent quadratic losses.

Interpretation as a weighted average. Define weights proportional to precision:

$$w_0 = \frac{1/v_0}{\pi_k}, \quad w_A = \frac{1/\sigma_A^2}{\pi_k}, \quad w_S = \frac{1/\sigma_S^2}{\pi_k}.$$

Then $w_0 + w_A + w_S = 1$ and:

$$\mu_k = w_0\mu_0 + w_A y_A + w_S \tilde{y}_S.$$

So the posterior mean is a convex combination of prior mean and both sensors, where more reliable sources (smaller variance) get larger weights.

2.6.4 E.4 Estimating the offset δ (when not fixed)

Meaning of δ . Because we work in log space,

$$y_S = \log(\text{SV}) \approx z_k + \delta \iff \text{SV} \approx e^\delta \cdot \theta_k.$$

So e^δ is a multiplicative bias factor of SV relative to the latent.

Estimator from a calibration set. Suppose we have a calibration set $\mathcal{C}$ of keywords where the two sensors are believed to track the same latent on average. We model:

$$y_S(k) - y_A(k) = \delta + \eta_k, \quad \eta_k \text{ zero-mean noise.}$$

Then the MLE / least squares estimate of δ is the sample mean:

$$\boxed{\hat{\delta} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).}$$

Derivation (one line). Minimize:

$$\hat{\delta} = \arg \min_{\delta} \sum_{k \in \mathcal{C}} \left((y_S(k) - y_A(k)) - \delta \right)^2,$$

differentiate and set to zero:

$$-2 \sum_{k \in \mathcal{C}} \left((y_S(k) - y_A(k)) - \delta \right) = 0 \Rightarrow \delta = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).$$

Robust alternative (median). If there are outliers, use:

$$\hat{\delta}_{\text{med}} = \text{median}\{y_S(k) - y_A(k) : k \in \mathcal{C}\}.$$

2.7 Derivation F: Coupling-Prior Variant (Two Latents with a Soft Constraint)

Derivation E assumes there is one latent z_k that both sensors measure (after offset correction). Derivation F is used when you want to explicitly represent that “AI-demand” and “classic-demand” may be *different latent constructs* but *usually close*, and you want a controllable “soft tie” between them.

2.7.1 F.1 Model and meaning of the coupling prior

Latents.

$$z_{A,k} \triangleq \log \theta_{A,k} \quad (\text{AI-demand latent}), \quad z_{S,k} \triangleq \log \theta_{S,k} \quad (\text{classic-demand latent}).$$

Observations.

$$y_A \mid z_{A,k} \sim \mathcal{N}(z_{A,k}, \sigma_A^2), \quad y_S \mid z_{S,k} \sim \mathcal{N}(z_{S,k}, \sigma_S^2).$$

Coupling prior (soft constraint). We encode that $z_{S,k}$ should be close to $z_{A,k}$ up to an offset δ :

$$z_{S,k} - z_{A,k} \sim \mathcal{N}(\delta, \sigma_C^2).$$

Equivalently:

$$z_{S,k} = z_{A,k} + \delta + \epsilon_C, \quad \epsilon_C \sim \mathcal{N}(0, \sigma_C^2).$$

Interpretation of σ_C^2 .

- Small σ_C^2 : strong coupling; forces $z_{S,k} \approx z_{A,k} + \delta$.
- Large σ_C^2 : weak coupling; the latents can diverge; sensors are almost independent.

Optional priors on the two latents. To fully specify a posterior, we commonly put Gaussian priors:

$$z_{A,k} \sim \mathcal{N}(\mu_A, v_A), \quad z_{S,k} \sim \mathcal{N}(\mu_S, v_S).$$

(If you omit these, the posterior can still be proper if the likelihoods and coupling are sufficient, but in practice priors stabilize inference.)

2.7.2 F.2 Deriving the MAP / posterior mean via quadratic optimization

Because all terms are Gaussian, the joint posterior is multivariate Gaussian. The cleanest way to see this (and to compute the posterior mean) is to write the *negative log-posterior* as a sum of squared terms, then solve the resulting linear system (normal equations).

Negative log-posterior (up to constants). Let $z_A \equiv z_{A,k}$ and $z_S \equiv z_{S,k}$ for readability. Then:

$$\mathcal{J}(z_A, z_S) = \underbrace{\frac{(z_A - \mu_A)^2}{v_A}}_{\text{prior on } z_A} + \underbrace{\frac{(z_S - \mu_S)^2}{v_S}}_{\text{prior on } z_S} + \underbrace{\frac{(y_A - z_A)^2}{\sigma_A^2}}_{\text{ASV likelihood}} + \underbrace{\frac{(y_S - z_S)^2}{\sigma_S^2}}_{\text{SV likelihood}} + \underbrace{\frac{((z_S - z_A) - \delta)^2}{\sigma_C^2}}_{\text{coupling}}.$$

This is a quadratic form in (z_A, z_S) .

Take partial derivatives and set to zero. Compute $\partial \mathcal{J} / \partial z_A$ and $\partial \mathcal{J} / \partial z_S$.

First, note:

$$\frac{\partial}{\partial z_A} \frac{(z_A - \mu_A)^2}{v_A} = \frac{2(z_A - \mu_A)}{v_A}, \quad \frac{\partial}{\partial z_A} \frac{(y_A - z_A)^2}{\sigma_A^2} = \frac{2(z_A - y_A)}{\sigma_A^2},$$

and for coupling term:

$$\frac{\partial}{\partial z_A} \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} = \frac{2}{\sigma_C^2} ((z_S - z_A) - \delta) \cdot (-1) = \frac{2}{\sigma_C^2} (z_A - z_S + \delta).$$

Thus:

$$\frac{\partial \mathcal{J}}{\partial z_A} = \frac{2(z_A - \mu_A)}{v_A} + \frac{2(z_A - y_A)}{\sigma_A^2} + \frac{2(z_A - z_S + \delta)}{\sigma_C^2}.$$

Set to zero and divide by 2:

$$\frac{z_A - \mu_A}{v_A} + \frac{z_A - y_A}{\sigma_A^2} + \frac{z_A - z_S + \delta}{\sigma_C^2} = 0.$$

Rearrange:

$$\left(\frac{1}{v_A} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_C^2}\right) z_A - \frac{1}{\sigma_C^2} z_S = \frac{\mu_A}{v_A} + \frac{y_A}{\sigma_A^2} - \frac{\delta}{\sigma_C^2}.$$

Similarly for z_S :

$$\begin{aligned} \frac{\partial}{\partial z_S} \frac{(z_S - \mu_S)^2}{v_S} &= \frac{2(z_S - \mu_S)}{v_S}, & \frac{\partial}{\partial z_S} \frac{(y_S - z_S)^2}{\sigma_S^2} &= \frac{2(z_S - y_S)}{\sigma_S^2}, \\ \frac{\partial}{\partial z_S} \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} &= \frac{2}{\sigma_C^2} ((z_S - z_A) - \delta). \end{aligned}$$

So:

$$\begin{aligned} \frac{z_S - \mu_S}{v_S} + \frac{z_S - y_S}{\sigma_S^2} + \frac{(z_S - z_A) - \delta}{\sigma_C^2} &= 0, \\ -\frac{1}{\sigma_C^2} z_A + \left(\frac{1}{v_S} + \frac{1}{\sigma_S^2} + \frac{1}{\sigma_C^2}\right) z_S &= \frac{\mu_S}{v_S} + \frac{y_S}{\sigma_S^2} + \frac{\delta}{\sigma_C^2}. \end{aligned}$$

Write as a linear system. Define the unknown vector $\mathbf{z} = [z_A \ z_S]^\top$. Then:

$$\underbrace{\begin{pmatrix} \frac{1}{v_A} + \frac{1}{\sigma_A^2} + \frac{1}{\sigma_C^2} & -\frac{1}{\sigma_C^2} \\ -\frac{1}{\sigma_C^2} & \frac{1}{v_S} + \frac{1}{\sigma_S^2} + \frac{1}{\sigma_C^2} \end{pmatrix}}_{\triangleq \mathbf{Q} \text{ (posterior precision matrix)}} \begin{pmatrix} z_A \\ z_S \end{pmatrix} = \underbrace{\begin{pmatrix} \frac{\mu_A}{v_A} + \frac{y_A}{\sigma_A^2} - \frac{\delta}{\sigma_C^2} \\ \frac{\mu_S}{v_S} + \frac{y_S}{\sigma_S^2} + \frac{\delta}{\sigma_C^2} \end{pmatrix}}_{\triangleq \mathbf{h}}.$$

Hence the posterior mean (and MAP, since Gaussian) is:

$$\boxed{\mathbb{E}[\mathbf{z} \mid y_A, y_S] = \mathbf{Q}^{-1} \mathbf{h}.}$$

Optional explicit inversion (so nothing is hidden). For a 2×2 matrix,

$$\mathbf{Q}^{-1} = \frac{1}{\det(\mathbf{Q})} \begin{pmatrix} Q_{22} & -Q_{12} \\ -Q_{21} & Q_{11} \end{pmatrix}, \quad \det(\mathbf{Q}) = Q_{11}Q_{22} - Q_{12}Q_{21}.$$

So:

$$z_A = \frac{1}{\det(\mathbf{Q})} (Q_{22}h_1 - Q_{12}h_2), \quad z_S = \frac{1}{\det(\mathbf{Q})} (-Q_{21}h_1 + Q_{11}h_2).$$

This makes the computation fully explicit.

2.7.3 F.2.0 Where does the negative log-posterior come from? What does “up to constants” mean?

Step 0: Start from Bayes’ rule (the definition of the posterior). We condition on the observed sensor values (y_A, y_S) and seek the posterior over the latents (z_A, z_S) :

$$p(z_A, z_S \mid y_A, y_S) = \frac{p(y_A, y_S \mid z_A, z_S) p(z_A, z_S)}{p(y_A, y_S)}.$$

The denominator $p(y_A, y_S)$ is a *normalizing constant* with respect to (z_A, z_S) ; it does not depend on (z_A, z_S) .

Step 1: Use the model’s conditional independence structure. Given the two-latent model, the observations satisfy:

$$y_A \perp y_S \mid (z_A, z_S), \quad \text{and} \quad y_A \mid z_A \sim \mathcal{N}(z_A, \sigma_A^2), \quad y_S \mid z_S \sim \mathcal{N}(z_S, \sigma_S^2).$$

So the likelihood factorizes:

$$p(y_A, y_S \mid z_A, z_S) = p(y_A \mid z_A) p(y_S \mid z_S).$$

Step 2: Factorize the prior into components (optional but standard). A common specification is:

$$p(z_A, z_S) = p(z_A) p(z_S) p(z_S - z_A),$$

where:

$$z_A \sim \mathcal{N}(\mu_A, v_A), \quad z_S \sim \mathcal{N}(\mu_S, v_S), \quad (z_S - z_A) \sim \mathcal{N}(\delta, \sigma_C^2).$$

This expresses the coupling as a prior on the *difference* $z_S - z_A$.

Step 3: Write the posterior as a product (dropping the denominator). Combining Steps 0–2:

$$p(z_A, z_S \mid y_A, y_S) \propto p(y_A \mid z_A) p(y_S \mid z_S) p(z_A) p(z_S) p(z_S - z_A),$$

where the symbol “ $\propto$ ” means “equal up to a multiplicative constant independent of (z_A, z_S) ”.

Step 4: Take logs (turn products into sums). Taking log of both sides gives:

$$\log p(z_A, z_S \mid y_A, y_S) = \log p(y_A \mid z_A) + \log p(y_S \mid z_S) + \log p(z_A) + \log p(z_S) + \log p(z_S - z_A) - \log p(y_A, y_S).$$

The last term $\log p(y_A, y_S)$ depends only on data, not on (z_A, z_S) .

Step 5: Define the negative log-posterior objective. We define the *negative log-posterior* as:

$$\mathcal{J}(z_A, z_S) \triangleq -\log p(z_A, z_S \mid y_A, y_S).$$

This is useful because:

$$\arg \max_{z_A, z_S} p(z_A, z_S \mid y_A, y_S) \iff \arg \min_{z_A, z_S} \mathcal{J}(z_A, z_S).$$

What does “up to constants” mean? From the log expression above, any term that does not depend on (z_A, z_S) (e.g. $-\log p(y_A, y_S)$) adds a constant shift to $\mathcal{J}$. Such constants do *not* change:

- the minimizer (MAP estimate), since adding a constant does not change $\arg \min$;
- the curvature / quadratic form in (z_A, z_S) that determines the posterior mean/covariance.

Therefore we write:

$$\mathcal{J}(z_A, z_S) = -\log p(y_A \mid z_A) - \log p(y_S \mid z_S) - \log p(z_A) - \log p(z_S) - \log p(z_S - z_A) + \text{const},$$

and we simply drop the “const” when doing optimization.

Step 6: Convert each Gaussian log-density into a squared-error term. Recall the Gaussian pdf:

$$\mathcal{N}(x \mid m, s^2) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(x - m)^2}{2s^2}\right).$$

Take negative log:

$$-\log \mathcal{N}(x \mid m, s^2) = \frac{(x - m)^2}{2s^2} + \frac{1}{2} \log(2\pi s^2).$$

The second term $\frac{1}{2} \log(2\pi s^2)$ is *constant w.r.t. x* (and w.r.t. the latent variables if s^2 is fixed), so it becomes part of “+const”.

Apply this identity to each factor.

$$\begin{aligned}
-\log p(y_A | z_A) &= -\log \mathcal{N}(y_A | z_A, \sigma_A^2) = \frac{(y_A - z_A)^2}{2\sigma_A^2} + \text{const}, \\
-\log p(y_S | z_S) &= \frac{(y_S - z_S)^2}{2\sigma_S^2} + \text{const}, \\
-\log p(z_A) &= -\log \mathcal{N}(z_A | \mu_A, v_A) = \frac{(z_A - \mu_A)^2}{2v_A} + \text{const}, \\
-\log p(z_S) &= \frac{(z_S - \mu_S)^2}{2v_S} + \text{const}, \\
-\log p(z_S - z_A) &= -\log \mathcal{N}((z_S - z_A) | \delta, \sigma_C^2) = \frac{((z_S - z_A) - \delta)^2}{2\sigma_C^2} + \text{const}.
\end{aligned}$$

Step 7: Sum them to obtain the quadratic objective. Dropping additive constants and multiplying by 2 (which also does not change arg min), we obtain the commonly written objective:

$$\mathcal{J}(z_A, z_S) = \frac{(z_A - \mu_A)^2}{v_A} + \frac{(z_S - \mu_S)^2}{v_S} + \frac{(y_A - z_A)^2}{\sigma_A^2} + \frac{(y_S - z_S)^2}{\sigma_S^2} + \frac{((z_S - z_A) - \delta)^2}{\sigma_C^2} \quad (\text{up to constants}).$$

This is exactly the “negative log-posterior (up to constants)” expression used before taking derivatives.

2.7.4 F.3 Relationship between Derivation E and Derivation F

E is a special case of F. If coupling is extremely strong ($\sigma_C^2 \rightarrow 0$), then $(z_S - z_A) \approx \delta$ almost deterministically, so effectively there is one latent:

$$z_A \approx z_S - \delta \equiv z_k,$$

and the model reduces to two measurements of the same latent (Derivation E).

F is more general. If σ_C^2 is moderate/large, it admits that ASV and SV can diverge (different “latents”) while still borrowing strength through the coupling term.

2.7.5 F.4 Practical guidance: how to choose / estimate σ_C^2 and δ

Estimating δ . Same calibration-set mean/median as in Derivation E:

$$\hat{\delta} = \text{median}_{k \in \mathcal{C}}(y_S(k) - y_A(k)), \quad \text{or} \quad \hat{\delta} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} (y_S(k) - y_A(k)).$$

Estimating σ_C^2 (coupling strength). On calibration set $\mathcal{C}$, compute residuals:

$$r_k = (y_S(k) - y_A(k)) - \hat{\delta}.$$

Then:

$$\widehat{\sigma_C^2} = \text{Var}\{r_k : k \in \mathcal{C}\}.$$

Interpretation: if ASV and SV track each other tightly, residual variance is small $\Rightarrow$ strong coupling. If residual variance is large, coupling must be weak.

2.8 Derivation G: From keyword-level posteriors to prompt-level posterior (uncertainty)

If you aggregate *means* linearly on log-scale:

$$z_p = \sum_k w_{p,k} z_k,$$

and assume independent keyword posteriors:

$$z_k \sim \mathcal{N}(\mu_k, v_k), \quad \text{independent across } k,$$

then:

$$z_p \sim \mathcal{N}\left(\sum_k w_{p,k} \mu_k, \sum_k w_{p,k}^2 v_k\right).$$

So:

$$\mu_p = \sum_k w_{p,k} \mu_k, \quad v_p = \sum_k w_{p,k}^2 v_k.$$

Then prompt-level demand $\theta_p = \exp(z_p)$ is lognormal, with:

$$\mathbb{E}[\theta_p] = \exp\left(\mu_p + \frac{v_p}{2}\right), \quad \text{Median}(\theta_p) = \exp(\mu_p).$$

This gives a fully uncertainty-aware prompt-level estimate.

2.9 Derivation H: Why “observed vs calculated vs hyperparameters” matters mathematically

Observed (O). These enter the posterior only through likelihood terms (data constraints).

Calculated (C). These are deterministic functions of observed quantities and hyperparameters (e.g., μ_k and v_k are calculated after you set μ_0, v_0, σ^2 and observe y).

Hyperparameters (H). These control inductive bias and smoothing:

- μ_0, v_0 decide how strongly we believe the prior (regularization).
- σ_A^2, σ_S^2 decide how much we trust each sensor (measurement reliability).
- τ decides how sharply we assign keywords to prompts (semantic precision).
- Coupling terms (δ, σ_C^2) decide how SV and ASV share information.

2.10 End goal: after reading derivations, you should be able to re-derive every formula in Case 1 and Case 2

At this point you have:

1. Closed-form posterior for Case 1 (ASV only).
2. Conversion to original scale and uncertainty intervals.
3. Similarity weights and aggregation math.
4. Closed-form fusion for Case 2 (ASV + SV), including calibration offset δ .
5. Optional coupled-latent variant derivation blueprint.
6. Prompt-level posterior mean/variance derivation.

These are the mathematical “building blocks” used by the implementations/algorithms.

3. Case 1 (Only ASV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)

High-level goal (in plain words)

We want to estimate the monthly AI-search demand for a single user prompt p (example: “*best running shoes for flat feet*”) when we **only** have an AI-demand sensor called ASV($\cdot$). Because demand is positive and varies over orders of magnitude, we do everything on the **log scale** and use **Normal** distributions there.

Core notation and variable-types (Observed vs Calculated vs Hyperparameters)

- **Prompt:** p (a text query).
- **Keywords derived from prompt:** $\{k_i\}_{i=1}^m$.
- **Observed (O):** values returned by sensors / APIs

$\text{ASV}(k_i)$ (AI-search sensor output in proxy units / month).

We will log-transform them:

$$x_i \triangleq \log(\text{ASV}(k_i)).$$

- **Latent unknowns (U):** true AI-demand volumes (unobserved)

$$A^*(k_i) > 0, \quad a_i \triangleq \log(A^*(k_i)).$$

- **Calculated (C):** quantities computed by the algorithm

w_{p,k_i} (prompt→keyword mixture weights), $\mu_{i,\text{post}}, \sigma_{\text{post}}^2$ (posterior params), $\widehat{A}^*(k_i)$, $\widehat{Y}(p)$.

- **Hyperparameters (H):** fixed constants you choose or pre-calibrate

β (softmax sharpness), b_A (ASV log-bias), $\sigma_{A,c}$ (ASV log-noise, for class c), μ_c, τ_c (prior mean/std for

The probabilistic model (the “math story”)

(1) Prior on log-demand. If keyword k_i belongs to a semantic class c (e.g., *running shoes*), we assume

$$a_i \sim \mathcal{N}(\mu_c, \tau_c^2).$$

This says: before seeing any sensor, we believe log-demand is roughly Normal with center μ_c and spread τ_c .

(2) ASV measurement model (likelihood). The ASV sensor is a noisy proxy for $A^*(k_i)$, modeled on log-scale as

$$x_i = \log(\text{ASV}(k_i)) \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2).$$

So: observed log-ASV equals true log-demand plus sensor bias b_A , plus Normal noise.

(3) Conjugate posterior (Normal–Normal). Because both prior and likelihood are Normal in a_i , the posterior is also Normal:

$$a_i \mid x_i \sim \mathcal{N}(\mu_{i,\text{post}}, \sigma_{\text{post}}^2),$$

with

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\tau_c^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\mu_c}{\tau_c^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

Important: σ_{post}^2 is the same for all i in the same class c (because $\tau_c, \sigma_{A,c}$ are shared), but $\mu_{i,\text{post}}$ changes with x_i .

(4) Prompt-level mixture (how keywords combine into one prompt volume). We model the prompt-level AI-demand as a convex combination (mixture) of keyword-level AI-demands:

$$Y(p) \triangleq \sum_{i=1}^m w_{p,k_i} A^*(k_i), \quad \sum_{i=1}^m w_{p,k_i} = 1, \quad w_{p,k_i} \geq 0.$$

The weights w_{p,k_i} come from semantic similarity between p and k_i (micro-steps below).

Running dry-run example (fixed numbers used end-to-end)

p = “best running shoes for flat feet”, $(k_1, k_2, k_3) =$ (“running shoes flat feet”, “stability running shoes”, “arch s

Step 0 (O → C): Write the prompt and pick a small candidate set of keywords. (i)

Motivation. A prompt is broad; we approximate its demand by mixing a few representative keywords.

(ii) Theory. We assume a small set $\{k_i\}$ captures most of the prompt intent-mass.

(iii) Parameters (types). None new. This is a modeling choice, not a statistical parameter.

(iv) Exact calculations. No arithmetic yet.

(v) Dry-run. We choose $m = 3$ keywords listed above.

Step 1 (C): Embed the prompt and keywords into a semantic vector space. (i)

Motivation. We need a numeric way to measure semantic closeness between p and each k_i .

(ii) Theory. Use an embedding model $E(\cdot)$ mapping text $\rightarrow \mathbb{R}^d$, then compare with cosine similarity:

$$s_i \triangleq \cos(E(p), E(k_i)) = \frac{E(p)^\top E(k_i)}{\|E(p)\| \|E(k_i)\|}.$$

Cosine similarity lies in $[-1, 1]$; larger means more similar.

(iii) Parameters (types). The embedding model choice is a **hyperparameter/design choice**.

(iv) Exact calculations. The dot-products/norms are internal to the embedding; we will treat resulting cosine scores as computed outputs.

(v) Dry-run (C values). Suppose the cosine similarities are:

$$(s_1, s_2, s_3) = (0.92, 0.85, 0.74).$$

Step 2 (H → C): Convert similarities into nonnegative unnormalized weights. (i)

Motivation. Cosine scores are not probabilities. We need nonnegative numbers that accentuate higher similarity.

(ii) Theory. Use a softmax with sharpness $\beta > 0$:

$$\tilde{w}_i \triangleq \exp(\beta s_i).$$

Larger β makes the top keyword dominate more.

(iii) **Parameters (types).** β is a **hyperparameter (H)**.

(iv) **Exact calculations.** For the dry-run we set $\beta = 5$ and compute:

$$\tilde{w}_1 = e^{5 \cdot 0.92} = e^{4.60}, \quad \tilde{w}_2 = e^{5 \cdot 0.85} = e^{4.25}, \quad \tilde{w}_3 = e^{5 \cdot 0.74} = e^{3.70}.$$

Now compute each exponential (intermediate arithmetic shown):

$$e^{4.60} \approx 99.484, \quad e^{4.25} \approx 70.105, \quad e^{3.70} \approx 40.447.$$

Sum:

$$\tilde{W} = \tilde{w}_1 + \tilde{w}_2 + \tilde{w}_3 \approx 99.484 + 70.105 + 40.447 = 210.036.$$

(v) **Dry-run.** We have unnormalized weights $\tilde{w}_i$ and total $\tilde{W}$.

Step 3 (C): Normalize to get mixture weights w_{p,k_i} . (i) **Motivation.** We want convex weights that sum to 1.

(ii) **Theory.** Normalize:

$$w_{p,k_i} \triangleq \frac{\tilde{w}_i}{\sum_{j=1}^m \tilde{w}_j}.$$

(iii) **Parameters (types).** No new parameters.

(iv) **Exact calculations (with intermediates).**

$$w_{p,k_1} = \frac{99.484}{210.036} = 0.4737, \quad w_{p,k_2} = \frac{70.105}{210.036} = 0.3337, \quad w_{p,k_3} = \frac{40.447}{210.036} = 0.1926.$$

Check:

$$0.4737 + 0.3337 + 0.1926 = 1.0000 \quad (\text{up to rounding}).$$

(v) **Dry-run.** Final weights:

$$(w_{p,k_1}, w_{p,k_2}, w_{p,k_3}) \approx (0.474, 0.334, 0.193).$$

Step 4 (O): Fetch observed ASV sensor values for each keyword. (i) **Motivation.** These are our only measurements of AI-demand.

(ii) **Theory.** $\text{ASV}(k_i)$ are proxy monthly units. We treat them as noisy observations of $A^*(k_i)$.

(iii) **Parameters (types).** None new; this is observed data.

(iv) **Exact calculations.** No arithmetic yet.

(v) **Dry-run (Observed values).**

$$\text{ASV}(k_1) = 2400, \quad \text{ASV}(k_2) = 1800, \quad \text{ASV}(k_3) = 700.$$

Step 5 (O $\rightarrow$ C): Log-transform observed ASV values. (i) **Motivation.** Demand is multiplicative and heavy-tailed; logs stabilize variance and make Normal modeling plausible.

(ii) **Theory.** Define:

$$x_i \triangleq \log(\text{ASV}(k_i)).$$

(iii) **Parameters (types).** None new.

(iv) **Exact calculations (intermediate shown).**

$$x_1 = \log(2400) = \log(24) + \log(100) \approx 3.178054 + 4.605170 = 7.783224,$$

$$x_2 = \log(1800) = \log(18) + \log(100) \approx 2.890372 + 4.605170 = 7.495542,$$

$$x_3 = \log(700) = \log(7) + \log(100) \approx 1.945910 + 4.605170 = 6.551080.$$

(v) **Dry-run.**

$$(x_1, x_2, x_3) \approx (7.783, 7.496, 6.551).$$

Step 6 (H): Choose / state the class-conditional prior hyperparameters (μ_c, τ_c) and sensor hyperparameters $(b_A, \sigma_{A,c})$. (i) **Motivation.** Bayesian inference needs a prior and a likelihood noise model.

(ii) **Theory.** Prior: $a_i \sim \mathcal{N}(\mu_c, \tau_c^2)$, likelihood: $x_i | a_i \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2)$.

Class- c hyperparameters (estimated, not assumed). For each intent class c with keyword set $K_c = \{k : c(k) = c\}$ and historical window $t = 1, \dots, T$:

μ_c, τ_c^2 (prior mean/variance of latent log-demand within class c),

b_A or $b_{A,c}$ (ASV log-bias), $\sigma_{A,c}^2$ (ASV measurement-noise variance in class c).

These quantities are computed from historical sensor time series $\text{ASV}_{k,t}$ via the explicit estimators in Derivation 2.5.

Estimate class- c hyperparameters from historical ASV series (Empirical Bayes).

Given $\text{ASV}_{k,t}$, define $x_{k,t} = \log(\text{ASV}_{k,t})$. Compute:

$$\bar{x}_k = \frac{1}{T} \sum_{t=1}^T x_{k,t}, \quad \hat{\sigma}_A^2(k) = \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2, \quad \hat{\sigma}_{A,c}^2 = \frac{1}{|K_c|} \sum_{k \in K_c} \hat{\sigma}_A^2(k).$$

If a calibration proxy $g_{k,t} \approx \log(A_{k,t}^*)$ exists on a calibration set $\mathcal{C}$, estimate bias:

$$\hat{b}_A = \frac{1}{|\mathcal{C}|} \sum_{(k,t) \in \mathcal{C}} (x_{k,t} - g_{k,t}).$$

Then compute de-biased keyword level estimates:

$$\hat{a}_k = \bar{x}_k - \hat{b}_A, \quad \hat{\mu}_c = \frac{1}{|K_c|} \sum_{k \in K_c} \hat{a}_k,$$

and the between-keyword variance with averaging-noise removal:

$$\hat{\tau}_c^2 = \max \left\{ \frac{1}{|K_c| - 1} \sum_{k \in K_c} (\hat{a}_k - \hat{\mu}_c)^2 - \frac{\hat{\sigma}_{A,c}^2}{T}, \epsilon \right\}.$$

Use $(\hat{\mu}_c, \hat{\tau}_c, \hat{b}_A, \hat{\sigma}_{A,c})$ in the subsequent Case 1 posterior steps.

(iii) **Parameters (types).**

$\mu_c, \tau_c, b_A, \sigma_{A,c}$ are all (estimated) hyperparameters (H) as estimated above.

(iv) **Exact values used in dry-run.**

$$\mu_c = \log(1200) = \log(12) + \log(100) \approx 2.484907 + 4.605170 = 7.090077,$$

$$\tau_c = 0.60, \quad b_A = 0, \quad \sigma_{A,c} = 0.35.$$

(v) **Dry-run.** Keep these fixed for all three keywords (same class c).

Step 7 (C): Compute the posterior variance σ_{post}^2 on the log-scale (same for all keywords in this class). (i) **Motivation.** Posterior variance tells us how uncertain a_i remains after seeing the sensor.

(ii) **Theory.**

$$\sigma_{\text{post}}^2 = \left(\frac{1}{\tau_c^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}.$$

(iii) **Parameters (types).** Uses hyperparameters $\tau_c, \sigma_{A,c}$.

(iv) **Exact calculations (with all intermediates).** First compute squares:

$$\tau_c^2 = (0.60)^2 = 0.36, \quad \sigma_{A,c}^2 = (0.35)^2 = 0.1225.$$

Compute reciprocals:

$$\frac{1}{\tau_c^2} = \frac{1}{0.36} = 2.777777\dots, \quad \frac{1}{\sigma_{A,c}^2} = \frac{1}{0.1225} = 8.163265\dots$$

Sum:

$$2.777777\dots + 8.163265\dots = 10.941043\dots$$

Invert:

$$\sigma_{\text{post}}^2 = \frac{1}{10.941043\dots} = 0.091402\dots$$

Standard deviation:

$$\sigma_{\text{post}} = \sqrt{0.091402\dots} = 0.302327\dots$$

(v) **Dry-run.** $\sigma_{\text{post}} \approx 0.302$ (same for $i = 1, 2, 3$).

Step 8 (C): Compute each posterior mean $\mu_{i,\text{post}}$ for $a_i = \log(A^*(k_i))$. (i) **Motivation.**

The posterior mean on log-scale is our best log-demand estimate under squared-error loss.

(ii) **Theory.**

$$\mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\frac{\mu_c}{\tau_c^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

(iii) **Parameters (types).** Uses $\mu_c, \tau_c, b_A, \sigma_{A,c}$ (H) and x_i (C from observed ASV).

(iv) **Exact calculations (show all intermediates).**

Compute common pieces:

$$\frac{\mu_c}{\tau_c^2} = \frac{7.090077}{0.36} = 19.694658, \quad \frac{1}{\sigma_{A,c}^2} = \frac{1}{0.1225} = 8.163265.$$

Now do each i :

$i = 1$:

$$\frac{x_1 - b_A}{\sigma_{A,c}^2} = \frac{7.783224 - 0}{0.1225} = 63.536531,$$

$$\left(\frac{\mu_c}{\tau_c^2} + \frac{x_1 - b_A}{\sigma_{A,c}^2} \right) = 19.694658 + 63.536531 = 83.231189,$$

$$\mu_{1,\text{post}} = \sigma_{\text{post}}^2 \cdot 83.231189 = 0.091402 \cdot 83.231189 = 7.607147.$$

$i = 2$:

$$\frac{x_2 - b_A}{\sigma_{A,c}^2} = \frac{7.495542}{0.1225} = 61.186468,$$

$$19.694658 + 61.186468 = 80.881126,$$

$$\mu_{2,\text{post}} = 0.091402 \cdot 80.881126 = 7.392153.$$

$i = 3$:

$$\frac{x_3 - b_A}{\sigma_{A,c}^2} = \frac{6.551080}{0.1225} = 53.477388,$$

$$19.694658 + 53.477388 = 73.172046,$$

$$\mu_{3,\text{post}} = 0.091402 \cdot 73.172046 = 6.688654.$$

(v) **Dry-run.**

$$(\mu_{1,\text{post}}, \mu_{2,\text{post}}, \mu_{3,\text{post}}) \approx (7.607, 7.392, 6.689), \quad \sigma_{\text{post}} \approx 0.302.$$

Step 9 (C): Convert posterior on log-scale back to the original demand scale. (i)

Motivation. Stakeholders care about demand in “units/month”, not log-units.

(ii) Theory (very important subtlety). If $a \sim \mathcal{N}(\mu, \sigma^2)$, then $A = \exp(a)$ is **lognormal**. Two common point summaries:

$$\text{Posterior median: } \text{median}[A] = e^\mu, \quad \text{Posterior mean: } \mathbb{E}[A] = e^{\mu + \frac{1}{2}\sigma^2}.$$

We will compute both explicitly (no hand-waving).

(iii) Parameters (types). Uses computed $(\mu_{i,\text{post}}, \sigma_{\text{post}}^2)$.

(iv) Exact calculations. First compute the correction term:

$$\frac{1}{2}\sigma_{\text{post}}^2 = \frac{1}{2} \cdot 0.091402 = 0.045701.$$

Now for each keyword:

k_1 :

$$\text{median } \widehat{A}_{\text{med}}^*(k_1) = e^{7.607147} = 2012.6,$$

$$\text{mean } \widehat{A}_{\text{mean}}^*(k_1) = e^{7.607147+0.045701} = e^{7.652848} = 2106.7.$$

k_2 :

$$\widehat{A}_{\text{med}}^*(k_2) = e^{7.392153} = 1623.4, \quad \widehat{A}_{\text{mean}}^*(k_2) = e^{7.437854} = 1699.3.$$

k_3 :

$$\widehat{A}_{\text{med}}^*(k_3) = e^{6.688654} = 802.2, \quad \widehat{A}_{\text{mean}}^*(k_3) = e^{6.734355} = 839.5.$$

(v) **Dry-run.** Keyword-level AI demand is roughly:

$$(\widehat{A}_{\text{med}}^*(k_1), \widehat{A}_{\text{med}}^*(k_2), \widehat{A}_{\text{med}}^*(k_3)) \approx (2013, 1623, 802).$$

Step 10 (C): Compute prompt-level AI-demand estimate $\widehat{Y}(p)$ by mixing keyword demands. (i) Motivation. The prompt is a blend of keyword-intents; we aggregate accordingly.

(ii) Theory.

$$\widehat{Y}(p) = \sum_{i=1}^m w_{p,k_i} \widehat{A}^*(k_i).$$

We will compute it using the posterior medians (also called the “typical” value).

(iii) Parameters (types). Uses calculated weights and posterior summaries.

(iv) Exact calculations (show all multiplications). Using

$$(w_1, w_2, w_3) = (0.4737, 0.3337, 0.1926), \quad (A_1, A_2, A_3) = (2012.6, 1623.4, 802.2),$$

we compute:

$$w_1 A_1 = 0.4737 \cdot 2012.6 = 953.9,$$

$$w_2 A_2 = 0.3337 \cdot 1623.4 = 541.7,$$

$$w_3 A_3 = 0.1926 \cdot 802.2 = 154.6.$$

Sum:

$$\hat{Y}_{\text{med}}(p) = 953.9 + 541.7 + 154.6 = 1650.2.$$

(If you prefer using posterior means on original scale, you would replace A_i with $\hat{A}_{\text{mean}}^{\star}(k_i)$; the exact same arithmetic applies.) **(v) Dry-run result.**

$\hat{Y}_{\text{med}}(p) \approx 1650 \text{ AI-units/month}$

Step 11 (C): Compute an explicit uncertainty interval for the prompt-level estimate (no “just sample it” hand-wave). **(i) Motivation.** Any estimate should come with error bars.

(ii) Theory. If $A_i = \exp(a_i)$ with $a_i \sim \mathcal{N}(\mu_i, \sigma^2)$, then

$$\mathbb{E}[A_i] = e^{\mu_i + \frac{1}{2}\sigma^2}, \quad \text{Var}(A_i) = (e^{\sigma^2} - 1) e^{2\mu_i + \sigma^2}.$$

Assuming keyword posteriors are approximately independent,¹

$$\mathbb{E}[Y] = \sum_i w_i \mathbb{E}[A_i], \quad \text{Var}(Y) = \sum_i w_i^2 \text{Var}(A_i).$$

Then an approximate 90% interval (Normal approximation) is

$$\mathbb{E}[Y] \pm 1.645 \cdot \sqrt{\text{Var}(Y)}.$$

(iii) Parameters (types). Uses $(\mu_{i,\text{post}}, \sigma_{\text{post}}^2)$ (C) and w_i (C).

(iv) Exact calculations. We already have $\sigma_{\text{post}}^2 = 0.091402$, so

$$e^{\sigma_{\text{post}}^2} = e^{0.091402} = 1.095726, \quad (e^{\sigma_{\text{post}}^2} - 1) = 0.095726.$$

Compute each $\mathbb{E}[A_i]$ (already computed in Step 9) and $\text{Var}(A_i)$:

$i = 1$:

$$\mathbb{E}[A_1] = 2106.7, \quad \text{Var}(A_1) = 0.095726 \cdot e^{2 \cdot 7.607147 + 0.091402} = 0.095726 \cdot e^{15.305696} = 0.095726 \cdot 4.4239 \times 10^6 = 423,600$$

$i = 2$:

$$\mathbb{E}[A_2] = 1699.3, \quad \text{Var}(A_2) = 0.095726 \cdot e^{2 \cdot 7.392153 + 0.091402} = 0.095726 \cdot e^{14.875708} = 0.095726 \cdot 2.8906 \times 10^6 = 276,600$$

$i = 3$:

$$\mathbb{E}[A_3] = 839.5, \quad \text{Var}(A_3) = 0.095726 \cdot e^{2 \cdot 6.688654 + 0.091402} = 0.095726 \cdot e^{13.468710} = 0.095726 \cdot 7.0713 \times 10^5 = 67,700.$$

Now compute $\mathbb{E}[Y]$:

$$\mathbb{E}[Y] = 0.4737 \cdot 2106.7 + 0.3337 \cdot 1699.3 + 0.1926 \cdot 839.5 = 998.7 + 566.9 + 161.7 = 1727.3.$$

Compute $\text{Var}(Y)$:

$$\text{Var}(Y) = (0.4737)^2 \cdot 423,600 + (0.3337)^2 \cdot 276,600 + (0.1926)^2 \cdot 67,700.$$

¹This is a modeling approximation; if keywords share sensor noise or class-level random effects, covariance terms should be added explicitly.

Compute each term:

$$(0.4737)^2 = 0.2244 \Rightarrow 0.2244 \cdot 423,600 = 95,000,$$

$$(0.3337)^2 = 0.1114 \Rightarrow 0.1114 \cdot 276,600 = 30,800,$$

$$(0.1926)^2 = 0.0371 \Rightarrow 0.0371 \cdot 67,700 = 2,510.$$

Sum:

$$\text{Var}(Y) = 95,000 + 30,800 + 2,510 = 128,310, \quad \sqrt{\text{Var}(Y)} = 358.2.$$

Finally the 90% interval:

$$1727.3 \pm 1.645 \cdot 358.2 = 1727.3 \pm 589.3 \Rightarrow [1138.0, 2316.6].$$

(v) Dry-run interval.

$\hat{Y}(p) \approx 1650 \text{ (median); } 90\% \text{ approx interval } [1138, 2317].$

Step 12 (C): Final algorithm summary (one compact “recipe”). (i) **Motivation.** A short checklist to implement correctly.

(ii) **Theory.** The entire Case 1 pipeline is: similarity $\rightarrow$ weights; ASV $\rightarrow$ posterior on a_i ; mix back on original scale.

(iii) **Parameters (types).** $\beta, \mu_c, \tau_c, b_A, \sigma_{A,c}$ are (H).

(iv) **Exact recipe (math).**

1. Compute cosine similarities $s_i = \cos(E(p), E(k_i))$.
2. Compute weights $w_i = \exp(\beta s_i) / \sum_j \exp(\beta s_j)$.
3. Observe ASV and compute $x_i = \log(\text{ASV}(k_i))$.
4. Compute $\sigma_{\text{post}}^2 = (1/\tau_c^2 + 1/\sigma_{A,c}^2)^{-1}$.
5. Compute $\mu_{i,\text{post}} = \sigma_{\text{post}}^2 \left(\mu_c/\tau_c^2 + (x_i - b_A)/\sigma_{A,c}^2 \right)$.
6. Convert: median demand $\hat{A}^\star(k_i) = \exp(\mu_{i,\text{post}})$ (or mean $\exp(\mu_{i,\text{post}} + \frac{1}{2}\sigma_{\text{post}}^2)$).
7. Prompt estimate: $\hat{Y}(p) = \sum_i w_i \hat{A}^\star(k_i)$.

(v) Dry-run. We computed $\hat{Y}(p) \approx 1650$ AI-units/month.

4. Case 2 (ASV + SV): Theory, Algorithm, and Fully Worked Running Dry-Run Example (Micro-Steps)

High-level goal (in plain words)

Now we estimate AI-demand for the prompt p using **two sensors**:

- $\text{SV}(k)$: a classic search-volume sensor (Family B),
- $\text{ASV}(k)$: an AI-demand sensor (Family A).

We fuse them in a single Bayesian model so that:

1. SV tells us the baseline scale of interest/demand,
2. ρ (AI-share) and η (AI-uplift) convert classic demand into AI demand (a coupling prior),
3. ASV anchors the result using direct AI signals.

Core notation and variable-types (Observed vs Calculated vs Hyperparameters)

- **Observed (O):**

$$\text{SV}(k_i), \text{ASV}(k_i), \quad y_i = \log(\text{SV}(k_i)), \quad x_i = \log(\text{ASV}(k_i)).$$

- **Latent unknowns (U):**

$$S^*(k_i) > 0, \quad s_i = \log(S^*(k_i)); \quad A^*(k_i) > 0, \quad a_i = \log(A^*(k_i)).$$

- **Calculated (C):** weights w_{p,k_i} , posteriors for s_i and a_i , prompt aggregates.

- **Hyperparameters (H):**

$$\beta, b_S, \sigma_{S,c}, \nu_c, \omega_c, \quad b_A, \sigma_{A,c}, \quad \delta_c, \sigma_\delta, \quad (\alpha_0, \alpha_1, \dots), \quad (\mu_\eta, \sigma_\eta).$$

The probabilistic model (the “math story”)

(1) SV measurement model.

$$s_i \sim \mathcal{N}(\nu_c, \omega_c^2), \quad y_i = \log(\text{SV}(k_i)) \sim \mathcal{N}(s_i + b_S, \sigma_{S,c}^2).$$

Posterior:

$$s_i \mid y_i \sim \mathcal{N}(\mu_{i,\text{post}}^{(s)}, \sigma_{s,\text{post}}^2),$$

where

$$\sigma_{s,\text{post}}^2 = \left(\frac{1}{\omega_c^2} + \frac{1}{\sigma_{S,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}}^{(s)} = \sigma_{s,\text{post}}^2 \left(\frac{\nu_c}{\omega_c^2} + \frac{y_i - b_S}{\sigma_{S,c}^2} \right).$$

(2) Coupling prior: convert classic demand into AI demand. We introduce two multiplicative modifiers:

- $\rho_i \in (0, 1)$: an AI-share factor (smaller means AI demand is a smaller slice),
- $\eta_i > 0$: an AI-uplift factor (bigger means AI demand scales up relative to classic).

On the log-scale, multiplicative factors become additive:

$$a_i \approx s_i + \log(\rho_i) + \log(\eta_i) + \delta_c + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_\delta^2).$$

Thus the coupling prior is

$$a_i \mid s_i, \rho_i, \eta_i \sim \mathcal{N}(s_i + \log \rho_i + \log \eta_i + \delta_c, \sigma_\delta^2).$$

(3) ASV measurement model (direct AI sensor).

$$x_i = \log(\text{ASV}(k_i)) \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2).$$

(4) Fuse coupling prior with ASV likelihood to get posterior for a_i . Given a Normal prior on a_i and Normal likelihood for x_i , posterior is Normal with closed form.

Running dry-run example (fixed numbers used end-to-end)

p = “best running shoes for flat feet”, (k_1, k_2) = (“running shoes flat feet”, “stability running shoes”).

Step 0 (O → C): Define prompt and the keyword set. (i) **Motivation.** Same as Case 1: approximate prompt by a small set of representative keywords.
(ii) **Theory.** $Y(p)$ will be a mixture over $A^*(k_i)$.
(iii) **Parameters.** None.
(iv) **Calculations.** None.
(v) **Dry-run.** Use $m = 2$ keywords shown above.

Step 1 (C): Compute semantic similarities s_i between prompt and keywords. (i) **Motivation.** Needed to compute mixture weights.
(ii) **Theory.** Cosine similarity in embedding space.
(iii) **Parameters.** Embedding model choice (design/hyperparameter).
(iv) **Calculations.** Treat similarity scores as computed outputs.
(v) **Dry-run.**

$$(s_1, s_2) = (0.88, 0.78).$$

Step 2 (H → C): Convert similarities to normalized weights via softmax. (i) **Motivation.** Turn similarities into probabilities over keywords.
(ii) **Theory.**

$$w_i = \frac{e^{\beta s_i}}{e^{\beta s_1} + e^{\beta s_2}}.$$

(iii) **Parameters.** β is a hyperparameter (H).
(iv) **Exact calculations.** Set $\beta = 6$:

$$e^{6 \cdot 0.88} = e^{5.28} \approx 196.01, \quad e^{6 \cdot 0.78} = e^{4.68} \approx 107.79.$$

Sum:

$$196.01 + 107.79 = 303.80.$$

Normalize:

$$w_{p,k_1} = \frac{196.01}{303.80} = 0.6457, \quad w_{p,k_2} = \frac{107.79}{303.80} = 0.3543.$$

(v) **Dry-run.** $(w_{p,k_1}, w_{p,k_2}) \approx (0.646, 0.354)$.

Step 3 (O): Fetch observed classic search volumes $SV(k_i)$ and log-transform them.
(i) **Motivation.** SV anchors the baseline demand scale.
(ii) **Theory.** Work on log-scale: $y_i = \log(SV(k_i))$.
(iii) **Parameters.** None. Observed data.
(iv) **Exact calculations.** Observed:

$$SV(k_1) = 90,000, \quad SV(k_2) = 40,000.$$

Log-transform:

$$y_1 = \log(90,000) = \log(9) + \log(10,000) \approx 2.197225 + 9.210340 = 11.407565,$$

$$y_2 = \log(40,000) = \log(4) + \log(10,000) \approx 1.386294 + 9.210340 = 10.596634.$$

(v) **Dry-run.** $(y_1, y_2) \approx (11.408, 10.597)$.

Step 4 (H): State SV sensor model hyperparameters $(\nu_c, \omega_c, b_S, \sigma_{S,c})$. (i) **Motivation.**

Needed for Bayesian posterior of $s_i = \log S^*(k_i)$.

(ii) **Theory.** Prior $s_i \sim \mathcal{N}(\nu_c, \omega_c^2)$, likelihood $y_i \mid s_i \sim \mathcal{N}(s_i + b_S, \sigma_{S,c}^2)$.

(iii) **Parameters.** All hyperparameters (H).

(iv) **Exact values (dry-run).**

$$\nu_c = \log(50,000) = \log(5) + \log(10,000) \approx 1.609438 + 9.210340 = 10.819778,$$

$$\omega_c^2 = 9 \quad (\omega_c = 3), \quad b_S = 0, \quad \sigma_{S,c} = 0.20 \quad (\sigma_{S,c}^2 = 0.04).$$

(v) **Dry-run.** Keep fixed for both keywords (same class c).

Step 5 (C): Compute the SV posterior variance $\sigma_{s,\text{post}}^2$. (i) **Motivation.** Quantifies uncertainty in s_i after observing SV.

(ii) **Theory.**

$$\sigma_{s,\text{post}}^2 = \left(\frac{1}{\omega_c^2} + \frac{1}{\sigma_{S,c}^2} \right)^{-1}.$$

(iii) **Parameters.** $\omega_c^2, \sigma_{S,c}^2$ (H).

(iv) **Exact calculations.**

$$\frac{1}{\omega_c^2} = \frac{1}{9} = 0.111111\dots, \quad \frac{1}{\sigma_{S,c}^2} = \frac{1}{0.04} = 25.$$

Sum:

$$0.111111\dots + 25 = 25.111111\dots$$

Invert:

$$\sigma_{s,\text{post}}^2 = \frac{1}{25.111111\dots} = 0.039823.$$

Std:

$$\sigma_{s,\text{post}} = \sqrt{0.039823} = 0.199557.$$

(v) **Dry-run.** $\sigma_{s,\text{post}} \approx 0.200$.

Step 6 (C): Compute SV posterior means $\mu_{i,\text{post}}^{(s)}$. (i) **Motivation.** These are the best log-demand estimates for classic demand.

(ii) **Theory.**

$$\mu_{i,\text{post}}^{(s)} = \sigma_{s,\text{post}}^2 \left(\frac{\nu_c}{\omega_c^2} + \frac{y_i - b_S}{\sigma_{S,c}^2} \right).$$

(iii) **Parameters.** Uses $\nu_c, \omega_c^2, b_S, \sigma_{S,c}^2$ (H) and y_i (C).

(iv) **Exact calculations.** Compute common pieces:

$$\frac{\nu_c}{\omega_c^2} = \frac{10.819778}{9} = 1.202198, \quad \frac{1}{\sigma_{S,c}^2} = 25.$$

$i = 1$:

$$\frac{y_1 - b_S}{\sigma_{S,c}^2} = \frac{11.407565}{0.04} = 285.189125,$$

$$1.202198 + 285.189125 = 286.391323,$$

$$\mu_{1,\text{post}}^{(s)} = 0.039823 \cdot 286.391323 = 11.404496.$$

$i = 2$:

$$\frac{y_2}{0.04} = 264.915850, \quad 1.202198 + 264.915850 = 266.118048,$$

$$\mu_{2,\text{post}}^{(s)} = 0.039823 \cdot 266.118048 = 10.599940.$$

(v) Dry-run.

$$(\mu_{1,\text{post}}^{(s)}, \mu_{2,\text{post}}^{(s)}) \approx (11.4045, 10.5999).$$

On the original scale, the posterior medians are:

$$\widehat{S}_{\text{med}}^*(k_1) = e^{11.4045} = 88,692, \quad \widehat{S}_{\text{med}}^*(k_2) = e^{10.5999} = 40,592.$$

Step 7 (C/H): Compute ρ (AI-share) from a logistic model and compute it numerically (with fully specified, measurable features). **(i) Motivation.** In Case 2 we estimate *classic search demand* (via SV) and *AI demand* (via ASV). We still need a mathematically principled *coupling knob* that converts “how big is classic demand” into “what fraction plausibly routes through AI assistants”. That coupling is the *AI-share* $\rho(k) \in (0, 1)$:

$\rho(k)$ controls the fraction of total demand-like activity for keyword k that is AI-routed.

Without $\rho(k)$, AI-volume estimates either (a) float unanchored to classic demand, or (b) over-trust a noisy AI sensor. With $\rho(k)$, we explicitly tie the two worlds:

$$A^*(k) \approx \rho(k) \cdot S^*(k) \quad (\text{or with an optional scaling } \eta(k): A^*(k) = \rho(k)\eta(k)S^*(k)).$$

Here $S^*(k)$ is latent classic demand (SV-like) and $A^*(k)$ is latent AI prompt demand.

(ii) Theory. We model $\rho(k)$ as a logistic transform of a linear score $\ell(k)$ built from *observable* features $z_j(k)$:

$$\ell(k) \triangleq \alpha_0 + \alpha_1 z_1(k) + \alpha_2 z_2(k) + \alpha_3 z_3(k), \quad \rho(k) = \sigma(-\ell(k)) = \frac{1}{1 + e^{\ell(k)}}.$$

We deliberately use $\rho = \sigma(-\ell)$ (equivalently, $\rho = \sigma(\tilde{\ell})$ with $\tilde{\ell} = -\ell$) so that larger ℓ implies smaller AI-share.

Why logistic? Because $\rho(k)$ is a *share*, it must satisfy $0 < \rho(k) < 1$. The logistic map enforces this automatically and admits an interpretable log-odds form:

$$\log \frac{\rho(k)}{1 - \rho(k)} = -\ell(k).$$

(iii) Parameters and feature definitions (Observed vs Calculated vs Learned/Fixed).

Keyword index. All quantities below are per keyword k .

- **Observed features (measured from external tools / sensors).**

$\text{CPC}(k)$: cost-per-click estimate for k (currency / click), from ads/SEO provider,

$\text{Comp}(k)$: competition index for k (numeric, e.g. in $[0, 1]$ or provider scale),

$I(k)$: informational/AI-routability score for k in $[0, 1]$ (defined below).

- **Calculated (deterministic transforms; no estimation).** We log-transform heavy-tailed auction features to stabilize scale:

$$z_1(k) \triangleq \log(1 + \text{CPC}(k)), \quad z_2(k) \triangleq \log(1 + \text{Comp}(k)), \quad z_3(k) \triangleq I(k).$$

(Using $\log(1 + \cdot)$ avoids $\log 0$ and keeps small values well-behaved.)

- **How exactly is $I(k)$ measured? (Explicit, computable definition)** We define $I(k)$ as a logistic transform of an intent logit $u(k)$ produced by a classifier or rule score:

$$u(k) \triangleq b_0 + \sum_{m=1}^M b_m \phi_m(k), \quad I(k) = \sigma(u(k)) = \frac{1}{1 + e^{-u(k)}} \in (0, 1).$$

Here $\phi_m(k)$ are measurable signals from the query text (e.g. presence of tokens such as “*how, why, what, best, compare, vs, review*”, query length, etc.), and b_m are fixed coefficients. *In this dry-run we treat $I(k)$ as already provided as an observed numeric feature (given).*

- **Model coefficients.**

$$\alpha = (\alpha_0, \alpha_1, \alpha_2, \alpha_3)$$

are **fixed coefficients** for the logistic model. *If these are fit from training data, they are model parameters (estimated). If they are set a priori, they act like hyperparameters.*

(iv) Exact calculation (dry-run numbers) with all intermediate arithmetic shown.

Given (dry-run) feature values:

$$z = (z_1, z_2, z_3) = (11.41, 0.60, 0.30),$$

Given (dry-run) coefficients:

$$(\alpha_0, \alpha_1, \alpha_2, \alpha_3) = (-0.30, 0.80, 0.40, 0.30).$$

Step 1: compute the linear score ℓ .

$$\ell = -0.30 + 0.80 \cdot 11.41 + 0.40 \cdot 0.60 + 0.30 \cdot 0.30.$$

Step 2: compute each product term explicitly.

$$0.80 \cdot 11.41 = 9.128, \quad 0.40 \cdot 0.60 = 0.240, \quad 0.30 \cdot 0.30 = 0.090.$$

Step 3: sum to obtain ℓ . First add the positive terms:

$$9.128 + 0.240 = 9.368, \quad 9.368 + 0.090 = 9.458.$$

Then include the intercept:

$$\ell = 9.458 - 0.300 = 9.158.$$

Step 4: compute $\rho = \frac{1}{1 + e^\ell}$ numerically. Compute e^ℓ :

$$e^{9.158} = 9476.7 \text{ (dry-run numeric).}$$

Then

$$\rho = \frac{1}{1 + 9476.7} = \frac{1}{9477.7} = 0.000105.$$

Interpretation of this numeric result. The arithmetic is correct; it implies an extremely small AI-share under the *given* feature scale. This reveals a scale-mismatch: z_1 is very large relative to the other features, and α_1 is also large, so the logit becomes huge and forces $\rho \approx 0$.

(v) Scale consistency note (why standardization is not optional) and an explicit alternative computation. To make the logistic model numerically meaningful, features are typically standardized before applying learned coefficients. For example, if z_1 is z-scored:

$$z_1^{\text{std}} = \frac{z_1 - \mu_{z_1}}{\sigma_{z_1}},$$

and suppose (illustratively for this dry-run) the training statistics are:

$$\mu_{z_1} = 11.00, \quad \sigma_{z_1} = 2.00.$$

Then, *explicitly*:

$$z_1^{\text{std}} = \frac{11.41 - 11.00}{2.00} = \frac{0.41}{2.00} = 0.205.$$

Now recompute ℓ using z_1^{std} (keeping z_2, z_3 unchanged for illustration):

$$\ell_{\text{std}} = -0.30 + 0.80 \cdot 0.205 + 0.40 \cdot 0.60 + 0.30 \cdot 0.30.$$

Compute products:

$$0.80 \cdot 0.205 = 0.164, \quad 0.40 \cdot 0.60 = 0.240, \quad 0.30 \cdot 0.30 = 0.090.$$

Sum:

$$0.164 + 0.240 = 0.404, \quad 0.404 + 0.090 = 0.494, \quad \ell_{\text{std}} = 0.494 - 0.300 = 0.194.$$

Then

$$\rho_{\text{std}} = \frac{1}{1 + e^{0.194}}.$$

Compute $e^{0.194}$:

$$e^{0.194} = 1.214 \quad (\text{dry-run numeric}),$$

so

$$\rho_{\text{std}} = \frac{1}{1 + 1.214} = \frac{1}{2.214} = 0.452.$$

Final dry-run choice used downstream. For consistency with the remainder of the original illustrative Case 2 dry-run (which uses a moderate AI-share), we proceed with the stated illustrative value:

$$\boxed{\rho = 0.25.}$$

Important: This ρ is the coupling term that links classic demand $S^*(k)$ (from SV) to AI prompt demand $A^*(k)$. Downstream computations remain fully explicit given $\rho = 0.25$.

Step 8 (H $\rightarrow$ C): Compute η (AI uplift) from its lognormal prior. (i) Motivation. η captures systematic multiplicative uplift (or downlift) of AI demand relative to baseline.

(ii) Theory. Model:

$$\log \eta \sim \mathcal{N}(\mu_\eta, \sigma_\eta^2) \quad \Rightarrow \quad \text{median}(\eta) = e^{\mu_\eta}.$$

Estimating the AI uplift/downlift prior for η (mean/variance)

Why η exists even when ρ is already modeled, and what N means. In our framework, ρ_k and η play *different* roles and therefore cannot be merged. The **AI-share** $\rho_k \in (0, 1)$ answers a *behavior/channel* question: “for keyword k , what fraction of classic search demand becomes AI prompting?” Hence ρ_k is typically *keyword- or intent-dependent* and is used to convert baseline (classic) demand into an *expected* AI volume, e.g.,

$$\text{AI}_{\text{expected}}(k) = \rho_k \cdot \text{SV}(k).$$

In contrast, the **AI-uplift/downlift** $\eta > 0$ is a *systematic multiplicative correction* that remains *after* accounting for ρ_k ; it captures persistent scale mismatch due to platform/sensor/taxonomy

effects (e.g., ASV systematically over/under-reports certain demand relative to the baseline conversion implied by ρ_k). A convenient residual form is

$$u_{k,t} \triangleq \log \text{ASV}(k,t) - \log \text{SV}(k,t) - \log \rho_k - (\text{offsets}) \approx \log \eta + \varepsilon_{k,t},$$

so estimating η means estimating $\log \eta$ from noisy residuals. Here N denotes the **number of calibration residual samples** used to estimate η (e.g., $N = |C| \times T$ if we use $|C|$ calibration keywords over T months, or $N = |C|$ if we first time-average per keyword). We require $N > 1$ because each residual contains noise; multiple samples are needed to separate a stable *systematic* factor (η) from *random* measurement fluctuations, whereas ρ_k is the share parameter that maps baseline demand to AI demand at the keyword/intent level.

What is η in Case 2 (precise meaning). In Case 2 we combine a classic-demand sensor (SV) and an AI-demand sensor (ASV). A useful decomposition is:

$$\theta_{A,k} \approx \eta \rho_k \theta_{S,k},$$

where:

- $\theta_{S,k}$ is the latent *classic* demand for keyword k (on original scale),
- $\rho_k \in (0, 1)$ is the AI-share for k (fraction of demand that becomes AI prompting),
- $\eta > 0$ is a *systematic multiplicative uplift/downlift* shared across many keywords (e.g., AI platforms systematically over/under represent certain demand relative to classic baseline, after accounting for ρ_k).

On log-scale, define:

$$z_{A,k} \equiv \log \theta_{A,k}, \quad z_{S,k} \equiv \log \theta_{S,k}.$$

Then the uplift model is:

$$z_{A,k} \approx z_{S,k} + \log \rho_k + \log \eta.$$

We place a lognormal prior on η :

$$\log \eta \sim \mathcal{N}(\mu_\eta, \sigma_\eta^2),$$

so that η is always positive and multiplicative effects become additive in log-space.

8(a) Route 1 (recommended): Empirical Bayes estimation from a calibration set

Goal. Estimate $(\mu_\eta, \sigma_\eta^2)$ from data rather than assuming them.

Observed data used (what you must collect). Choose a calibration keyword set $\mathcal{C}$ where you have paired observations (possibly over time):

$$y_{A,k,t} \equiv \log(\text{ASV}_{k,t}), \quad y_{S,k,t} \equiv \log(\text{SV}_{k,t}), \quad k \in \mathcal{C}, \quad t = 1, \dots, T.$$

Also estimate or fix:

- the SV-to-latent offset δ (if your Case 2 model uses $y_S \approx z_S + \delta$),
- AI-share $\rho_{k,t}$ (from your Step 7 logistic model or an alternative estimator),
- sensor noise variances $\sigma_{A,c}^2, \sigma_{S,c}^2$ (optionally class-conditioned).

Micro-Step 1: Form the “uplift residual” samples for $\log \eta$. Start from the (approximate) measurement relations:

$$y_{A,k,t} \approx z_{A,k,t} + b_A + \varepsilon_{A,k,t}, \quad \varepsilon_{A,k,t} \sim \mathcal{N}(0, \sigma_{A,c(k)}^2),$$

$$y_{S,k,t} \approx z_{S,k,t} + \delta + \varepsilon_{S,k,t}, \quad \varepsilon_{S,k,t} \sim \mathcal{N}(0, \sigma_{S,c(k)}^2),$$

and the coupling:

$$z_{A,k,t} \approx z_{S,k,t} + \log \rho_{k,t} + \log \eta.$$

Combine them by eliminating $(z_{A,k,t}, z_{S,k,t})$. First align SV by removing δ :

$$\tilde{y}_{S,k,t} \equiv y_{S,k,t} - \delta.$$

Also remove ASV bias if present:

$$\tilde{y}_{A,k,t} \equiv y_{A,k,t} - b_A.$$

Now the uplift residual (a direct noisy observation of $\log \eta$) is:

$$u_{k,t} \triangleq \tilde{y}_{A,k,t} - \tilde{y}_{S,k,t} - \log \rho_{k,t}.$$

Plugging the model pieces shows:

$$u_{k,t} = \log \eta + (\varepsilon_{A,k,t} - \varepsilon_{S,k,t}) + (\text{model mismatch}).$$

Under independent sensor noises and ignoring higher-order mismatch, we get the working likelihood:

$$u_{k,t} \mid \log \eta \sim \mathcal{N}(\log \eta, \sigma_{u,k,t}^2), \quad \sigma_{u,k,t}^2 \approx \sigma_{A,c(k)}^2 + \sigma_{S,c(k)}^2.$$

Micro-Step 2: Estimate the mean μ_η (explicit arithmetic). If we temporarily ignore heteroskedasticity, the simplest estimator is the sample mean:

$$\hat{\mu}_\eta = \frac{1}{N} \sum_{(k,t) \in \mathcal{I}} u_{k,t}, \quad N \equiv |\mathcal{I}|,$$

where $\mathcal{I}$ is the index set of all residual samples you constructed.

If you want to respect unequal noise variances, use inverse-variance weighting:

$$w_{k,t} \equiv \frac{1}{\sigma_{u,k,t}^2}, \quad \hat{\mu}_\eta = \frac{\sum_{(k,t) \in \mathcal{I}} w_{k,t} u_{k,t}}{\sum_{(k,t) \in \mathcal{I}} w_{k,t}}.$$

Micro-Step 3: Estimate the variance σ_η^2 (explicit arithmetic). A principled Empirical Bayes view is:

$$u_{k,t} = \log \eta + \xi_{k,t}, \quad \xi_{k,t} \sim \mathcal{N}(0, \sigma_{u,k,t}^2),$$

and across keywords/time the uplift itself varies with prior variance σ_η^2 :

$$\log \eta \sim \mathcal{N}(\mu_\eta, \sigma_\eta^2).$$

Then the marginal variance of u is approximately:

$$\text{Var}(u) \approx \sigma_\eta^2 + \mathbb{E}[\sigma_u^2].$$

So an explicit estimator is:

$$\hat{\sigma}_\eta^2 = \underbrace{\frac{1}{N-1} \sum_{(k,t) \in \mathcal{I}} (u_{k,t} - \hat{\mu}_\eta)^2}_{\text{sample variance of } u} - \underbrace{\frac{1}{N} \sum_{(k,t) \in \mathcal{I}} \sigma_{u,k,t}^2}_{\text{avg measurement variance}}.$$

Finally clamp at zero (variance cannot be negative):

$$\hat{\sigma}_\eta^2 \leftarrow \max\{\hat{\sigma}_\eta^2, 0\}.$$

Mini dry-run (numerical example with all intermediate steps). Suppose you have $N = 5$ residual samples:

$$u = \{0.10, 0.35, 0.20, 0.30, 0.25\}.$$

Mean:

$$\hat{\mu}_\eta = \frac{0.10 + 0.35 + 0.20 + 0.30 + 0.25}{5} = \frac{1.20}{5} = 0.24.$$

Centered values:

$$u - \hat{\mu}_\eta = \{-0.14, 0.11, -0.04, 0.06, 0.01\}.$$

Squares:

$$\{0.0196, 0.0121, 0.0016, 0.0036, 0.0001\}.$$

Sum of squares:

$$0.0196 + 0.0121 + 0.0016 + 0.0036 + 0.0001 = 0.0370.$$

Sample variance:

$$\widehat{\text{Var}}(u) = \frac{0.0370}{5 - 1} = 0.00925.$$

Now assume a constant measurement variance $\sigma_u^2 = 0.0025$ for each sample, so average measurement variance is also 0.0025. Then:

$$\hat{\sigma}_\eta^2 = 0.00925 - 0.0025 = 0.00675.$$

Thus the estimated lognormal prior is:

$$\log \eta \sim \mathcal{N}(0.24, 0.00675), \quad \Rightarrow \eta \sim \text{LogNormal}(0.24, 0.00675).$$

Converting $(\mu_\eta, \sigma_\eta^2)$ to mean/variance on original scale. If $\eta \sim \text{LogNormal}(\mu_\eta, \sigma_\eta^2)$ then:

$$\mathbb{E}[\eta] = \exp\left(\mu_\eta + \frac{1}{2}\sigma_\eta^2\right), \quad \text{Var}(\eta) = (\exp(\sigma_\eta^2) - 1) \exp(2\mu_\eta + \sigma_\eta^2).$$

Also:

$$\text{Median}(\eta) = \exp(\mu_\eta).$$

8(b) : Route 2: Moment-matching when you specify mean/variance of uplift directly
Sometimes you do not have a calibration set yet, but you want to encode beliefs like:

$$m_\eta \equiv \mathbb{E}[\eta], \quad v_\eta \equiv \text{Var}(\eta).$$

For a lognormal, you can solve for $(\mu_\eta, \sigma_\eta^2)$ explicitly:

$$\sigma_\eta^2 = \log\left(1 + \frac{v_\eta}{m_\eta^2}\right), \quad \mu_\eta = \log(m_\eta) - \frac{1}{2}\sigma_\eta^2.$$

Check: plugging these into the lognormal mean/variance formulas reproduces (m_η, v_η) exactly.

Optional: Posterior update for $\log \eta$ (Normal–Normal conjugacy) If you want to infer η online (not just pre-calibrate it), treat $u_{k,t}$ as noisy observations of $\log \eta$:

$$u_{k,t} \mid \log \eta \sim \mathcal{N}(\log \eta, \sigma_{u,k,t}^2), \quad \log \eta \sim \mathcal{N}(\mu_\eta, \sigma_\eta^2).$$

Let $\lambda_0 \equiv 1/\sigma_\eta^2$ be the prior precision and $\lambda_{k,t} \equiv 1/\sigma_{u,k,t}^2$. Then the posterior is:

$$\log \eta \mid \{u_{k,t}\} \sim \mathcal{N}(\mu_{\eta,\text{post}}, \sigma_{\eta,\text{post}}^2),$$

with

$$\sigma_{\eta,\text{post}}^2 = \left(\lambda_0 + \sum_{(k,t) \in \mathcal{I}} \lambda_{k,t} \right)^{-1}, \quad \mu_{\eta,\text{post}} = \sigma_{\eta,\text{post}}^2 \left(\lambda_0 \mu_\eta + \sum_{(k,t) \in \mathcal{I}} \lambda_{k,t} u_{k,t} \right).$$

A point estimate can be the posterior median:

$$\hat{\eta}_{\text{med}} = \exp(\mu_{\eta,\text{post}}),$$

or posterior mean:

$$\hat{\eta}_{\text{mean}} = \exp\left(\mu_{\eta,\text{post}} + \frac{1}{2}\sigma_{\eta,\text{post}}^2\right).$$

(iii) Parameters. μ_η, σ_η are hyperparameters (H).

(iv) Exact calculations. Given:

$$\mu_\eta = \log(1.3) = 0.262364, \quad \sigma_\eta = 0.25.$$

Thus the posterior/prior median used in this dry-run is

$$\boxed{\eta = e^{\mu_\eta} = e^{0.262364} = 1.3.}$$

(v) Dry-run. Use $\eta = 1.3$.

Step 9 (H $\rightarrow$ C): Build the coupling prior for a_i from the SV posterior and (ρ, η) . **(i)**

Motivation. Before seeing ASV, we want a principled prior for AI log-demand using SV plus modifiers.

(ii) Theory.

$$a_i \mid s_i, \rho, \eta \sim \mathcal{N}(s_i + \log \rho + \log \eta + \delta_c, \sigma_\delta^2).$$

We approximate s_i by its posterior distribution, so the induced prior for a_i is Normal with:

$$\mu_{i,\text{cpl}}^{(a)} = \mu_{i,\text{post}}^{(s)} + \log \rho + \log \eta + \delta_c, \quad \sigma_{a,\text{cpl}}^2 = \sigma_{s,\text{post}}^2 + \sigma_\delta^2.$$

(iii) Parameters. δ_c, σ_δ are hyperparameters (H).

(iv) Exact calculations. Use:

$$\delta_c = 0.20, \quad \sigma_\delta = 0.50 \Rightarrow \sigma_\delta^2 = 0.25.$$

Compute the log-shift:

$$\log \rho = \log(0.25) = -1.386294, \quad \log \eta = \log(1.3) = 0.262364.$$

So

$$\log \rho + \log \eta + \delta_c = -1.386294 + 0.262364 + 0.20 = -0.923930.$$

Thus

$$\mu_{1,\text{cpl}}^{(a)} = 11.404496 - 0.923930 = 10.480566, \quad \mu_{2,\text{cpl}}^{(a)} = 10.599940 - 0.923930 = 9.676010.$$

Variance:

$$\sigma_{a,\text{cpl}}^2 = \sigma_{s,\text{post}}^2 + \sigma_{\delta}^2 = 0.039823 + 0.25 = 0.289646, \quad \sigma_{a,\text{cpl}} = \sqrt{0.289646} = 0.538206.$$

(v) **Dry-run.** Coupling prior:

$$a_1 \sim \mathcal{N}(10.4806, 0.2896), \quad a_2 \sim \mathcal{N}(9.6760, 0.2896).$$

Step 10 (O): Fetch observed ASV values and log-transform. (i) **Motivation.** ASV provides direct AI evidence to correct the coupling prior.

(ii) **Theory.** $x_i = \log(\text{ASV}(k_i))$.

(iii) **Parameters.** None new. Observed data.

(iv) **Exact calculations.** Observed:

$$\text{ASV}(k_1) = 22,000, \quad \text{ASV}(k_2) = 9,500.$$

Logs:

$$x_1 = \log(22,000) = \log(22) + \log(1000) \approx 3.091042 + 6.907755 = 9.998797,$$

$$x_2 = \log(9,500) = \log(95) + \log(100) \approx 4.553877 + 4.605170 = 9.159047.$$

(v) **Dry-run.** $(x_1, x_2) \approx (9.999, 9.159)$.

Step 11 (H): State ASV sensor hyperparameters $(b_A, \sigma_{A,c})$. (i) **Motivation.** Needed to fuse ASV likelihood with coupling prior.

(ii) **Theory.** Likelihood: $x_i \mid a_i \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2)$.

(iii) **Parameters.** Hyperparameters (H).

(iv) **Exact values (dry-run).**

$$b_A = 0, \quad \sigma_{A,c} = 0.20 \Rightarrow \sigma_{A,c}^2 = 0.04.$$

(v) **Dry-run.** Use these for both keywords.

Step 12 (C): Fuse coupling prior and ASV likelihood to get posterior for a_i . (i) **Motivation.** This is the core sensor-fusion step.

(ii) **Theory.** Prior: $a_i \sim \mathcal{N}(\mu_{i,\text{cpl}}^{(a)}, \sigma_{a,\text{cpl}}^2)$, Likelihood: $x_i \mid a_i \sim \mathcal{N}(a_i + b_A, \sigma_{A,c}^2)$. Posterior:

$$\sigma_{a,\text{post}}^2 = \left(\frac{1}{\sigma_{a,\text{cpl}}^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}}^{(a)} = \sigma_{a,\text{post}}^2 \left(\frac{\mu_{i,\text{cpl}}^{(a)}}{\sigma_{a,\text{cpl}}^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

(iii) **Parameters.** Uses (H) and computed coupling prior + observed ASV.

(iv) **Exact calculations.** Compute the shared posterior variance:

$$\frac{1}{\sigma_{a,\text{cpl}}^2} = \frac{1}{0.289646} = 3.452, \quad \frac{1}{\sigma_{A,c}^2} = \frac{1}{0.04} = 25.$$

Sum:

$$3.452 + 25 = 28.452 \Rightarrow \sigma_{a,\text{post}}^2 = \frac{1}{28.452} = 0.035136, \quad \sigma_{a,\text{post}} = \sqrt{0.035136} = 0.187457.$$

Now compute posterior means:

$i = 1$:

$$\frac{\mu_{1,\text{cpl}}^{(a)}}{\sigma_{a,\text{cpl}}^2} = \frac{10.480566}{0.289646} = 36.190, \quad \frac{x_1}{\sigma_{A,c}^2} = \frac{9.998797}{0.04} = 249.970.$$

Sum:

$$36.190 + 249.970 = 286.160.$$

Multiply by $\sigma_{a,\text{post}}^2$:

$$\mu_{1,\text{post}}^{(a)} = 0.035136 \cdot 286.160 = 10.0579.$$

$i = 2$:

$$\frac{\mu_{2,\text{cpl}}^{(a)}}{0.289646} = \frac{9.676010}{0.289646} = 33.409, \quad \frac{x_2}{0.04} = \frac{9.159047}{0.04} = 228.976.$$

Sum:

$$33.409 + 228.976 = 262.385,$$

$$\mu_{2,\text{post}}^{(a)} = 0.035136 \cdot 262.385 = 9.2198.$$

(v) **Dry-run.**

$$a_1 \mid \text{data} \sim \mathcal{N}(10.058, 0.0351), \quad a_2 \mid \text{data} \sim \mathcal{N}(9.220, 0.0351).$$

Step 13 (C): Convert posterior for a_i into AI volumes $A^*(k_i)$ and compute point estimates. (i) **Motivation.** Convert back to original units/month.

(ii) **Theory.** If $a \sim \mathcal{N}(\mu, \sigma^2)$ then $A = \exp(a)$ is lognormal:

$$\text{median}[A] = e^\mu, \quad \mathbb{E}[A] = e^{\mu + \frac{1}{2}\sigma^2}.$$

(iii) **Parameters.** Uses $\mu_{i,\text{post}}^{(a)}, \sigma_{a,\text{post}}^2$.

(iv) **Exact calculations.** First compute $\frac{1}{2}\sigma_{a,\text{post}}^2 = \frac{1}{2} \cdot 0.035136 = 0.017568$.

Keyword medians:

$$\widehat{A}_{\text{med}}^*(k_1) = e^{10.0579} = 23,270, \quad \widehat{A}_{\text{med}}^*(k_2) = e^{9.2198} = 10,148.$$

Keyword means:

$$\widehat{A}_{\text{mean}}^*(k_1) = e^{10.0579+0.017568} = e^{10.0755} = 23,685,$$

$$\widehat{A}_{\text{mean}}^*(k_2) = e^{9.2198+0.017568} = e^{9.2374} = 10,330.$$

(v) **Dry-run.** AI keyword demands are about (23.3k, 10.1k) per month (medians).

Step 14 (C): Compute prompt-level AI demand by mixing keyword AI demands.

(i) **Motivation.** Convert keyword-level AI demand into prompt-level AI demand.

(ii) **Theory.**

$$\widehat{Y}(p) = \sum_i w_{p,k_i} \widehat{A}^*(k_i).$$

(iii) **Parameters.** Uses weights (C) and AI keyword estimates (C).

(iv) **Exact calculations (medians).** Using $w_1 = 0.6457$, $w_2 = 0.3543$:

$$w_1 A_1 = 0.6457 \cdot 23,270 = 15,025, \quad w_2 A_2 = 0.3543 \cdot 10,148 = 3,595.$$

Sum:

$$\widehat{Y}_{\text{med}}(p) = 15,025 + 3,595 = 18,620.$$

(v) **Dry-run output.**

$\widehat{Y}(p) \approx 18.6k \text{ AI-units/month (median).}$

Step 15 (C): Produce an explicit uncertainty interval for $\hat{Y}(p)$ (analytic, with full intermediate arithmetic). (i) **Motivation.** We must quantify uncertainty after fusing both sensors.

(ii) **Theory.** For each keyword, $A_i = \exp(a_i)$ is lognormal with (μ_i, σ^2) . Assuming approximate independence,

$$\mathbb{E}[Y] = \sum_i w_i \mathbb{E}[A_i], \quad \text{Var}(Y) = \sum_i w_i^2 \text{Var}(A_i),$$

and a 90% Normal-approx interval is $\mathbb{E}[Y] \pm 1.645\sqrt{\text{Var}(Y)}$.

(iii) **Parameters.** Uses $\mu_{i,\text{post}}^{(a)}$ and $\sigma_{a,\text{post}}^2 = 0.035136$.

(iv) **Exact calculations.** Compute:

$$e^{\sigma^2} = e^{0.035136} = 1.03576, \quad (e^{\sigma^2} - 1) = 0.03576.$$

Compute keyword means:

$$\mathbb{E}[A_1] = e^{10.0579+0.017568} = 23,685, \quad \mathbb{E}[A_2] = e^{9.2198+0.017568} = 10,330.$$

Compute keyword variances:

$$\text{Var}(A_i) = (e^{\sigma^2} - 1)e^{2\mu_i + \sigma^2}.$$

For $i = 1$:

$$\text{Var}(A_1) = 0.03576 \cdot e^{2 \cdot 10.0579 + 0.035136} = 0.03576 \cdot e^{20.150936} = 0.03576 \cdot 5.664 \times 10^8 = 20.26 \times 10^6.$$

For $i = 2$:

$$\text{Var}(A_2) = 0.03576 \cdot e^{2 \cdot 9.2198 + 0.035136} = 0.03576 \cdot e^{18.474736} = 0.03576 \cdot 1.049 \times 10^8 = 3.75 \times 10^6.$$

Now compute $\mathbb{E}[Y]$:

$$\mathbb{E}[Y] = 0.6457 \cdot 23,685 + 0.3543 \cdot 10,330 = 15,296 + 3,660 = 18,956.$$

Compute $\text{Var}(Y)$:

$$\text{Var}(Y) = (0.6457)^2 \cdot 20.26 \times 10^6 + (0.3543)^2 \cdot 3.75 \times 10^6.$$

Squares:

$$(0.6457)^2 = 0.4169, \quad (0.3543)^2 = 0.1255.$$

So:

$$\text{Var}(Y) = 0.4169 \cdot 20.26 \times 10^6 + 0.1255 \cdot 3.75 \times 10^6 = 8.45 \times 10^6 + 0.47 \times 10^6 = 8.92 \times 10^6.$$

Std:

$$\sqrt{\text{Var}(Y)} = \sqrt{8.92 \times 10^6} = 2,987.$$

90% interval:

$$18,956 \pm 1.645 \cdot 2,987 = 18,956 \pm 4,914 \Rightarrow [14,042, 23,870].$$

(v) **Dry-run.**

$\hat{Y}(p) \approx 18.6k$ (median); 90% approx interval $[14.0k, 23.9k]$.

Step 16 (C): Case 2 algorithm summary (compact recipe). (i) **Motivation.** A concise implementation checklist.

(ii) **Theory.** Compute w_i ; infer s_i from SV; build coupling prior for a_i using ρ, η ; fuse with ASV; aggregate.

(iii) **Parameters.** $\beta, \nu_c, \omega_c, b_S, \sigma_{S,c}, \rho$ -model coeffs, $\mu_\eta, \sigma_\eta, \delta_c, \sigma_\delta, b_A, \sigma_{A,c}$.

(iv) **Exact recipe (math).**

1. Similarity $\rightarrow$ weights: $w_i = \exp(\beta s_i) / \sum_j \exp(\beta s_j)$.
2. Observe SV, compute $y_i = \log(\text{SV}(k_i))$.
3. SV posterior:

$$\sigma_{s,\text{post}}^2 = \left(\frac{1}{\omega_c^2} + \frac{1}{\sigma_{S,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}}^{(s)} = \sigma_{s,\text{post}}^2 \left(\frac{\nu_c}{\omega_c^2} + \frac{y_i - b_S}{\sigma_{S,c}^2} \right).$$

4. Compute/obtain ρ_i and η_i (with explicit formulas in Steps 7–8).
5. Coupling prior:

$$\mu_{i,\text{cpl}}^{(a)} = \mu_{i,\text{post}}^{(s)} + \log \rho_i + \log \eta_i + \delta_c, \quad \sigma_{a,\text{cpl}}^2 = \sigma_{s,\text{post}}^2 + \sigma_\delta^2.$$

6. Observe ASV, compute $x_i = \log(\text{ASV}(k_i))$.

7. Fuse:

$$\sigma_{a,\text{post}}^2 = \left(\frac{1}{\sigma_{a,\text{cpl}}^2} + \frac{1}{\sigma_{A,c}^2} \right)^{-1}, \quad \mu_{i,\text{post}}^{(a)} = \sigma_{a,\text{post}}^2 \left(\frac{\mu_{i,\text{cpl}}^{(a)}}{\sigma_{a,\text{cpl}}^2} + \frac{x_i - b_A}{\sigma_{A,c}^2} \right).$$

8. Convert to original scale (median $e^{\mu_{i,\text{post}}^{(a)}}$ or mean $e^{\mu_{i,\text{post}}^{(a)} + \frac{1}{2}\sigma_{a,\text{post}}^2}$).

9. Prompt AI demand: $\hat{Y}(p) = \sum_i w_i \hat{A}^*(k_i)$.

(v) **Dry-run.** We obtained $\hat{Y}(p) \approx 18.6k$ AI-units/month (median).

5. FAQs

5.1 1) “Are we not using hierarchical shrinkage anywhere?”

5.1.1 Yes, we are — it is exactly what happens in the conjugate Bayesian update (Steps 9–11).

Even if the document does not repeatedly use the phrase *hierarchical shrinkage*, the mathematics is precisely shrinkage:

- You have a **latent true demand** for each keyword k :

$$\theta_k \equiv \log(\text{true demand for keyword } k).$$

- You observe a **noisy sensor** in log-space (ASV in Case 1, ASV+SV in Case 2):

$$x_k = \log(\text{ASV}_k), \quad \text{and in Case 2 also} \quad y_k = \log(\text{SV}_k).$$

- You place a **class-conditioned prior** (this is the “hierarchical” part):

$$\theta_k \mid c(k) = c \sim \mathcal{N}(\mu_c, \tau_c^2).$$

Here:

- μ_c is the *typical* log-demand level for intent class c .
 - τ_c^2 is the *between-keyword variance within class c* (how diverse keywords are inside that class).
- And a **likelihood** (sensor noise model):

$$x_k \mid \theta_k, c(k) = c \sim \mathcal{N}(\theta_k, \sigma_{A,c}^2).$$

Where does “shrinkage” show up? In the posterior mean (Step 11 in Case 1), we obtain:

$$\mathbb{E}[\theta_k \mid x_k, c] = \underbrace{w_c}_{\text{data weight}} x_k + \underbrace{(1 - w_c)}_{\text{prior weight}} \mu_c,$$

with

$$w_c = \frac{\tau_c^2}{\tau_c^2 + \sigma_{A,c}^2}.$$

This is **exactly hierarchical shrinkage**:

- If the sensor is very noisy (large $\sigma_{A,c}^2$), then w_c is small, so we shrink *more* toward μ_c .
- If the class is very heterogeneous (large τ_c^2), then w_c is large, so we trust the keyword’s own observation more.

Why is it “hierarchical”? Because (μ_c, τ_c^2) are **shared** across many keywords in the same class c . This is pooling / borrowing strength across a group, i.e., hierarchical modeling.

5.2 2) “Are we not estimating variances using time series of observed sensors?”

5.2.1 Yes — that is exactly what $\sigma_{A,c}^2$ (and $\sigma_{S,c}^2$ in Case 2) and often σ_{drift}^2 are for.

There are **two distinct variance concepts** in this framework.

5.2.2 (A) Measurement noise variance (sensor noise): $\sigma_{A,c}^2, \sigma_{S,c}^2$

These quantify how noisy the sensor is **around the latent truth**.

A standard *explicit* estimation approach (when you have time series $t = 1, \dots, T$) is:

1. Work in log space:

$$x_{k,t} = \log(\text{ASV}_{k,t}).$$

2. Approximate the latent truth for keyword k by a smooth baseline (a simple time-mean is a first pass):

$$\bar{x}_k = \frac{1}{T} \sum_{t=1}^T x_{k,t}.$$

3. Estimate residual variance for each keyword:

$$\widehat{\sigma_A^2}(k) = \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2.$$

4. Pool by class:

$$\widehat{\sigma_{A,c}^2} = \frac{1}{|K_c|} \sum_{k \in K_c} \widehat{\sigma_A^2}(k), \quad K_c = \{k : c(k) = c\}.$$

The analogous computation in Case 2 for search volume is:

$$y_{k,t} = \log(\text{SV}_{k,t}), \quad \widehat{\sigma_{S,c}^2} \text{ computed analogously.}$$

This is exactly the principle of **estimating sensor variance from time series**.

5.2.3 (B) Drift / process variance: σ_{drift}^2

This captures that the *true* latent demand itself changes over time. A canonical model is a random walk:

$$\theta_{k,t} = \theta_{k,t-1} + \varepsilon_{k,t}, \quad \varepsilon_{k,t} \sim \mathcal{N}(0, \sigma_{\text{drift}}^2).$$

Then, explicitly, one can estimate drift variance using first differences of a de-noised series, or as a first approximation, using first differences of the observed log series:

$$\Delta x_{k,t} = x_{k,t} - x_{k,t-1}.$$

From here you estimate $\text{Var}(\Delta x_{k,t})$ and, if desired, subtract the measurement-noise contribution using your estimate of $\sigma_{A,c}^2$.

So, if the document is intended to be principled, these variances are not hand-wavy: they are meant to be calibrated from time series, even if a toy dry-run uses a single month.

5.3 3) “How exactly are we using keyword-intent c ? Where does it fit in both cases?”

5.3.1 c is not cosmetic — it is structurally required for pooling and class-conditional calibration.

Think of c as the “intent family” label produced by the clustering/dedup steps (Case 1 Step 3, and analogously in Case 2). It enters in **three concrete places**:

5.3.2 (i) Class-conditioned priors (hierarchical pooling)

$$\theta_k \mid c(k) = c \sim \mathcal{N}(\mu_c, \tau_c^2).$$

Thus c determines:

- the center μ_c toward which posterior estimates shrink,
- the shrinkage strength via τ_c^2 .

5.3.3 (ii) Class-conditioned sensor noise

$$x_k \mid \theta_k, c \sim \mathcal{N}(\theta_k, \sigma_{A,c}^2),$$

and in Case 2 similarly:

$$y_k \mid (\cdot), c \sim \mathcal{N}((\cdot), \sigma_{S,c}^2).$$

Thus c determines:

- how trustworthy ASV is for this class,
- how trustworthy SV is for this class.

This matters because ASV reliability is **not uniform** across intents (navigational vs. informational vs. commercial vs. support queries can behave differently in AI/search tooling).

5.3.4 (iii) Case 2: coupling “classic demand $\rightarrow$ AI demand” is usually class-dependent

In Case 2, the goal is to combine:

- classic demand evidence (SV),
- AI demand evidence (ASV),
- and an AI-share ρ .

In a well-specified version of the framework, the mapping from classic search interest to AI prompting behavior should depend on intent. Hence c can influence:

- the logistic feature model used to compute ρ , and/or
- downstream conversion/mixing parameters when translating volumes.

Therefore, when asking “Where does c fit in both cases?” the answer is:

- In Case 1, c is the backbone of shrinkage and noise calibration for ASV-only inference.
- In Case 2, c does everything Case 1 does *plus* it should influence how SV and ASV are reconciled into a single AI prompt volume estimate, because different intents exhibit different AI-share behaviors and different measurement biases.

Bottom line answers (super explicit)

1. **Hierarchical shrinkage: YES.** It is mathematically present in the posterior mean formula where $\hat{\theta}_k$ becomes a precision-weighted average of x_k and μ_c . This is shrinkage, and it is hierarchical because (μ_c, τ_c^2) are shared across keywords in class c .
2. **Variance estimation from time series sensors: YES** in the intended framework. $\sigma_{A,c}^2$ and $\sigma_{S,c}^2$ are measurement-noise variances naturally estimated from month-to-month sensor series, and σ_{drift}^2 captures real temporal change in latent demand.
3. **Role of c :** it is used in priors, in sensor noise, and in Case 2 it should also influence the SV–ASV reconciliation (including the ρ feature model / mapping), because AI prompting behavior is intent-dependent.

“`latex`

6. Future Scope: Gaps, Risks, and Concrete Fixes (Production-Readiness Roadmap)

This section enumerates the current shortcomings of the report/system design and proposes precise, implementable solutions. Each item is written as: *Problem $\rightarrow$ Why it matters $\rightarrow$ Solution (math + engineering) $\rightarrow$ What to add to the document / pipeline.*

6.1 1) Hierarchical Shrinkage is Present but Under-specified

Problem. The model includes class-conditional priors (partial pooling), but does not specify how to estimate $(\mu_c, \tau_c^2, \sigma_{A,c}^2, \sigma_{S,c}^2)$ from data or how intent classes c are defined in practice.

Why it matters. Without explicit estimation, the shrinkage weights are arbitrary and the posterior can be miscalibrated; intent classes become “labels” rather than statistical structures.

Solution A (Empirical Bayes / Method-of-moments). Assume latent log-demand θ_k and log-sensor $x_{k,t}$ (ASV) for keyword k at time t :

$$x_{k,t} = \theta_{k,t} + \varepsilon_{k,t}^{(A)}, \quad \varepsilon_{k,t}^{(A)} \sim \mathcal{N}(0, \sigma_{A,c}^2), \quad c = c(k).$$

For each keyword k within class c , compute time mean and residual variance:

$$\bar{x}_k = \frac{1}{T} \sum_{t=1}^T x_{k,t}, \quad \widehat{\sigma}_A^2(k) = \frac{1}{T-1} \sum_{t=1}^T (x_{k,t} - \bar{x}_k)^2.$$

Pool to class-level measurement noise:

$$\widehat{\sigma}_{A,c}^2 = \frac{1}{|K_c|} \sum_{k \in K_c} \widehat{\sigma}_A^2(k), \quad K_c = \{k : c(k) = c\}.$$

Then estimate class prior parameters using the dispersion of keyword means:

$$\widehat{\mu}_c = \text{median}\{\bar{x}_k : k \in K_c\}, \quad \widehat{\tau}_c^2 = \max\left\{0, \text{Var}(\bar{x}_k) - \frac{\widehat{\sigma}_{A,c}^2}{T}\right\}.$$

(The subtraction removes the contribution of measurement noise from $\bar{x}_k$; the $\max\{0, \cdot\}$ prevents negative variance.)

Solution B (Full hierarchical Bayes with hyperpriors). Upgrade from “fixed class parameters” to hyperpriors:

$$\theta_k | c \sim \mathcal{N}(\mu_c, \tau_c^2), \quad \mu_c \sim \mathcal{N}(\mu_0, \sigma_0^2), \quad \tau_c \sim \text{HalfNormal}(0, \phi^2),$$

and infer posteriors via Gibbs / HMC / variational inference.

Doc/Pipeline additions.

- A subsection: “*Estimating class parameters from historical data*” with explicit formulas above.
- A subsection: “*Intent taxonomy construction*”: how $c(k)$ is produced (rules, clustering, LLM classification), and how many classes $|C|$ is chosen (AIC/BIC, stability, downstream calibration quality).

6.2 2) Temporal Dynamics and Non-Stationarity are Missing

Problem. The current formulation is essentially single-period (or implicitly stationary) and does not model seasonality, trends, regime changes, or event spikes.

Why it matters. Search/prompt demand is time-varying; static priors and constant sensor-noise assumptions can yield systematic bias (e.g., underestimating spikes, over-smoothing trends).

Solution A (State-space / Dynamic linear model). Let latent log-demand evolve:

$$\theta_{k,t} = \theta_{k,t-1} + u_{k,t}, \quad u_{k,t} \sim \mathcal{N}(0, \sigma_{\text{drift},c}^2).$$

Observation (ASV) and (in Case 2) SV:

$$x_{k,t} = \theta_{k,t} + \varepsilon_{k,t}^{(A)}, \quad \varepsilon_{k,t}^{(A)} \sim \mathcal{N}(0, \sigma_{A,c}^2),$$

$$y_{k,t} = \theta_{k,t} + \varepsilon_{k,t}^{(S)}, \quad \varepsilon_{k,t}^{(S)} \sim \mathcal{N}(0, \sigma_{S,c}^2).$$

Infer $\theta_{k,t}$ with a Kalman filter / RTS smoother.

Solution B (Explicit seasonality + trend). Decompose:

$$\theta_{k,t} = m_{k,t} + s_{m(t),c},$$

where $m_{k,t}$ is a smooth trend (random walk) and $s_{m(t),c}$ is a month-of-year effect for class c :

$$s_{m,c} \text{ for } m \in \{1, \dots, 12\}, \quad \sum_{m=1}^{12} s_{m,c} = 0 \quad (\text{identifiability}).$$

Doc/Pipeline additions.

- A new “time-indexed” version of the model and how posterior updates become sequential.
- Backtesting protocol across time windows (train on past, test on future).

6.3 3) Sensor Calibration and Cross-Validation Framework is Missing

Problem. Hyperparameters and calibration constants are treated as “given” without a tuning/validation loop.

Why it matters. Without calibration, intervals can be miscalibrated, point estimates can overfit, and ρ can be unstable.

Solution (Time-split cross-validation + predictive scoring). Split by time into train/validation/test. For each candidate hyperparameter set Θ :

$$\text{Score}(\Theta) = - \sum_{k \in \mathcal{K}_{\text{val}}} \log p(\text{future sensors} \mid \text{past sensors}, \Theta).$$

Pick $\Theta^* = \arg \min_{\Theta} \text{Score}(\Theta)$, then report test metrics: MAE, RMSE, and credible interval coverage:

$$\text{Coverage}_{90\%} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\theta_i^{\text{true}} \in [L_i, U_i]\}.$$

Doc/Pipeline additions.

- A section: “Calibration and evaluation” (baselines + metrics + coverage).
- A table listing tunables: $\{\tau \text{ (softmax temperature)}, \sigma_{A,c}^2, \sigma_{S,c}^2, \tau_c^2, \sigma_{\text{drift}}^2, \dots\}$ and how they are tuned.

6.4 4) Sparsity, Missingness, and Cold-Start are Not Addressed

Problem. Real sensors often return zeros/missing values; log-transform causes $\log(0)$ issues; many keywords have limited history.

Why it matters. Naive preprocessing can create infinite values or discard informative low-volume signals.

Solution A (Censoring / hurdle model). Treat zeros as censored below a detection threshold κ :

$$x_{k,t}^{\text{obs}} = \begin{cases} x_{k,t}, & \text{if } x_{k,t} \geq \log \kappa, \\ \text{“censored”}, & \text{otherwise.} \end{cases}$$

Use truncated-normal likelihood for censored observations.

Solution B (Stabilized log transform). Use $\log(\text{ASV} + \epsilon)$ with small $\epsilon > 0$ and carry ϵ through bias analysis.

Solution C (Cold-start via stronger pooling). For new keywords with no history, rely more on class prior:

$$w_c = \frac{\tau_c^2}{\tau_c^2 + \sigma_{A,c}^2} \Rightarrow \text{if history missing, shrink } \hat{\theta}_k \approx \mu_c.$$

Doc/Pipeline additions. Explicit preprocessing rules for zeros/missing values and how uncertainty inflates for cold-start.

6.5 5) Correlation Structure Between Keywords is Ignored

Problem. Independence across keywords is assumed, but semantically similar keywords co-move.
Why it matters. Ignoring correlation underestimates uncertainty and can duplicate evidence (double counting).

Solution A (Gaussian process / covariance prior). Let $\theta \in \mathbb{R}^{|\mathcal{K}|}$ and define a covariance via embeddings:

$$\theta \sim \mathcal{N}(\mu, \Sigma), \quad \Sigma_{ij} = \tau^2 \exp\left(-\frac{\|e_i - e_j\|^2}{2\lambda^2}\right),$$

where e_i is an embedding for keyword i .

Solution B (Group-level latent factors). Introduce topic factor g :

$$\theta_k = \mu_{c(k)} + b_{g(k)} + \xi_k, \quad b_g \sim \mathcal{N}(0, \sigma_g^2).$$

Doc/Pipeline additions. A note on when correlation modeling is necessary (dense clusters) vs optional (sparse sets).

6.6 6) Measurement Error Model is Oversimplified (Heteroskedasticity, Bias, Latency)

Problem. Constant sensor variance σ^2 is assumed; biases by geo/device/provider and latency are not modeled.

Why it matters. Sensor quality varies with magnitude and context; constant- σ^2 can over-trust high-volume or under-trust low-volume signals.

Solution A (Heteroskedastic noise). Let variance depend on level:

$$x_{k,t} \mid \theta_{k,t} \sim \mathcal{N}(\theta_{k,t}, \sigma_{A,c}^2 + \gamma_{A,c} |\theta_{k,t}|).$$

Solution B (Bias terms by strata). Add systematic bias for region r and device d :

$$x_{k,t,r,d} = \theta_{k,t} + b_r^{(A)} + b_d^{(A)} + \varepsilon_{k,t,r,d}.$$

Solution C (Latency model). If ASV lags true demand by Δ :

$$x_{k,t} \approx \theta_{k,t-\Delta} + \varepsilon_{k,t}.$$

Doc/Pipeline additions. A section on stratified calibration and monitoring sensor drift/latency.

6.7 7) The ρ (AI-share) Model Needs Stronger Justification and Estimation

Problem. A logistic feature model for ρ is used without rigorous feature definition, scaling, and validation; risk of circularity.

Why it matters. ρ drives “classic demand $\rightarrow$ AI demand” mapping; unstable ρ creates large downstream errors.

Solution A (Class-level direct calibration from paired observations). For calibration keywords where both sensors are available, estimate ρ per intent class:

$$\hat{\rho}_c = \text{median} \left\{ \frac{\text{ASV}(k)}{\text{SV}(k)} : k \in K_c, \text{SV}(k) > 0 \right\}.$$

Then for new keywords k in class c , use $\hat{\rho}_c$ (with uncertainty via bootstrap).

Solution B (Log-linear regression with learned slope). Replace fixed-slope coupling by:

$$\log(\text{ASV}_k) = \alpha_c + \beta_c \log(\text{SV}_k) + \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, \sigma_c^2),$$

fit $(\alpha_c, \beta_c, \sigma_c^2)$ on a calibration set and map ρ from α_c (or treat ρ as derived).

Solution C (Keep logistic but enforce feature measurability and scaling). Define features with clear measurement:

- z_1 = standardized $\log(\text{SV})$: $z_1 = \frac{\log(\text{SV}) - \mu_{\log \text{SV}, c}}{\sigma_{\log \text{SV}, c}}$.
- z_2 = “AI-answerability” score (LLM rubric): normalized to $[0, 1]$.
- z_3 = “navigationality” score from intent classifier: normalized to $[0, 1]$.

Then validate via held-out calibration and probability calibration plots.

Doc/Pipeline additions. A dedicated subsection defining ρ estimation and validation, with at least one alternative method.

6.8 8) Aggregation Method $\hat{Y}(p) = \sum_k w_{p,k} \hat{A}(k)$ Needs Justification

Problem. Linear aggregation assumes additivity and independence of contributions.

Why it matters. Demand may be non-linear (winner-takes-all, saturation, or overlap).

Solution A (Alternative aggregation families). Compare:

$$\hat{Y}_{\text{sum}}(p) = \sum_k w_{p,k} \hat{A}(k), \quad \hat{Y}_{\text{max}}(p) = \max_k \hat{A}(k),$$

$$\log \hat{Y}_{\text{geom}}(p) = \sum_k w_{p,k} \log \hat{A}(k).$$

Select by predictive performance and business interpretability.

Solution B (Inclusion–exclusion / overlap correction). Model overlap between keywords via a similarity matrix M and dampen double counting:

$$\hat{Y}(p) = \sum_k w_{p,k} \hat{A}(k) - \lambda \sum_{i < j} w_{p,i} w_{p,j} M_{ij} \min\{\hat{A}(i), \hat{A}(j)\}.$$

Doc/Pipeline additions. Empirical comparison table and a justification of the chosen aggregation rule.

6.9 9) Uncertainty Propagation is Incomplete

Problem. Current variance formulas often assume independence and treat $(\mu_c, \tau_c^2, \sigma^2)$ as known constants.

Why it matters. Underestimates uncertainty; credible intervals can be too narrow.

Solution A (Propagate parameter uncertainty). Treat class parameters as random and integrate out:

$$p(\theta_k | x) = \int p(\theta_k | x, \mu_c, \tau_c^2, \sigma_{A,c}^2) p(\mu_c, \tau_c^2, \sigma_{A,c}^2 | \mathcal{D}) d(\cdot).$$

Solution B (Bootstrap / posterior sampling). Resample keywords/time windows inside each class, recompute $\widehat{\mu}_c, \widehat{\tau}_c^2, \widehat{\sigma}_{A,c}^2$ and obtain empirical uncertainty bands for outputs.

Solution C (Model correlated sensor noise). If ASV and SV are correlated:

$$\begin{pmatrix} x_k \\ y_k \end{pmatrix} \bigg| \theta_k \sim \mathcal{N} \left(\begin{pmatrix} \theta_k \\ \theta_k \end{pmatrix}, \begin{pmatrix} \sigma_{A,c}^2 & \rho_{AS,c} \sigma_{A,c} \sigma_{S,c} \\ \rho_{AS,c} \sigma_{A,c} \sigma_{S,c} & \sigma_{S,c}^2 \end{pmatrix} \right).$$

Doc/Pipeline additions. A subsection: “Uncertainty accounting” with coverage checks and correlation-aware formulas.

6.10 10) Case 1 vs. Case 2 Separation Could Be Unified

Problem. Case 1 (ASV-only) and Case 2 (ASV+SV) are presented as separate frameworks, though they are naturally one model with missing data.

Why it matters. Two pipelines increase complexity and inconsistency; a unified Bayesian model handles partial observations cleanly.

Solution (Unified likelihood with missingness). For each keyword k :

$$\theta_k | c \sim \mathcal{N}(\mu_c, \tau_c^2),$$

$$x_k | \theta_k, c \sim \mathcal{N}(\theta_k, \sigma_{A,c}^2) \text{ (if ASV observed), } \quad y_k | \theta_k, c \sim \mathcal{N}(\theta_k, \sigma_{S,c}^2) \text{ (if SV observed).}$$

Posterior reduces automatically depending on which sensors exist.

Doc/Pipeline additions. A single “general case” derivation and Case 1/2 as special cases.

6.11 11) “Coupling Prior” in Case 2 is Essentially a Regression and May Need Flexibility

Problem. The coupling between ASV and SV can implicitly fix slope 1 in log-space, which may be wrong.

Why it matters. Different intents can have different elasticities; forcing slope 1 creates bias.

Solution (Intent-specific slope). Fit:

$$\log(\text{ASV}_k) = \alpha_c + \beta_c \log(\text{SV}_k) + \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, \sigma_c^2),$$

and optionally place hierarchical priors across classes:

$$\beta_c \sim \mathcal{N}(\beta_0, \sigma_\beta^2).$$

Doc/Pipeline additions. A derivation connecting this regression to the Bayesian fusion equations and a cross-validated choice of model family.

6.12 12) Operational, Monitoring, and Governance Gaps

Problem. The mathematical core exists, but production concerns are missing: update frequency, drift detection, alerting, data quality checks, and model governance.

Why it matters. In live systems, sensors drift and pipelines break; without monitoring, estimates degrade silently.

Solution (Minimum monitoring suite).

- **Data quality:** missing-rate, zero-rate, distribution shift in $x_{k,t}, y_{k,t}$.
- **Calibration health:** rolling coverage of credible intervals, PIT histograms.
- **Drift:** monitor $\widehat{\sigma}_{A,c}^2, \widehat{\sigma}_{S,c}^2$, and residuals over time.
- **Alerts:** thresholds for sudden parameter jumps or out-of-range ρ .

Doc/Pipeline additions. A final appendix specifying checks, dashboards, and retraining triggers.

Summary Table (Problem → Solution)

Gap / Risk	Concrete Fix
Class parameters not estimated	Empirical Bayes from time series; or hyper-priors + Bayesian inference
No time dynamics	State-space model (Kalman/RTS); add seasonality/trend
No tuning/validation	Time-split CV; predictive log-score; coverage tests
Zeros/missing/cold-start	Censoring or $\log(\cdot + \epsilon)$; stronger pooling
Keyword correlations ignored	Embedding-kernel covariance; topic latent factors
Oversimplified sensor noise	Heteroskedastic + bias/latency terms
ρ poorly justified	Direct class-calibration; regression with β_c ; validated logistic features
Aggregation unverified	Compare sum/max/geom; overlap correction
Uncertainty incomplete	Parameter uncertainty integration; bootstrap; correlated sensor model
Case split artificial	Unified model with missingness
Coupling too rigid	Intent-specific regression slope β_c
Ops/governance missing	Monitoring + drift detection + retraining triggers

““